SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Fundamentals Of Data Science **COURSE CODE: DSA0401**

COURSE OUTCOME COVERED

CO3: Evaluate datasets using descriptive statistics, including summarization, outlier detection, and distribution analysis, and perform statistical inference through estimation and hypothesis testing. (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 02 **Total Marks:100**

Annexure A

Data Interpretation

Code of Conduct:

I E.Jeffrey Samuels/192524412, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Jeffrey Samuels.E

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary) 5 Marks	Level 3 (Proficient) 3 /4 Marks	Level 2 (Developing) 2 Marks	Level 1 (Beginning) 1 Mark	Marks
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data	
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided	
Application of Concepts	20%	Effectively applies relevant concepts/ tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data	
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions	
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand	

Annexure A

Problem 01: Hospital Patient Health Monitoring

Scenario

A hospital wants to investigate how patient lifestyle factors affect blood pressure levels.

Dataset

Patient_ID	Daily_Exercise (Minutes)	BMI	Systolic_BP
P001	10	32	150
P002	15	31	148
P003	20	30	145
P004	25	29	142
P005	30	28	138
P006	35	27	134
P007	40	26	130
P008	45	25	126
P009	50	24	122
P010	55	23	118

Tasks

1. Find the mean and standard deviation of blood pressure.
2. Calculate covariance between:
    - Exercise and Blood Pressure

- BMI and Blood Pressure

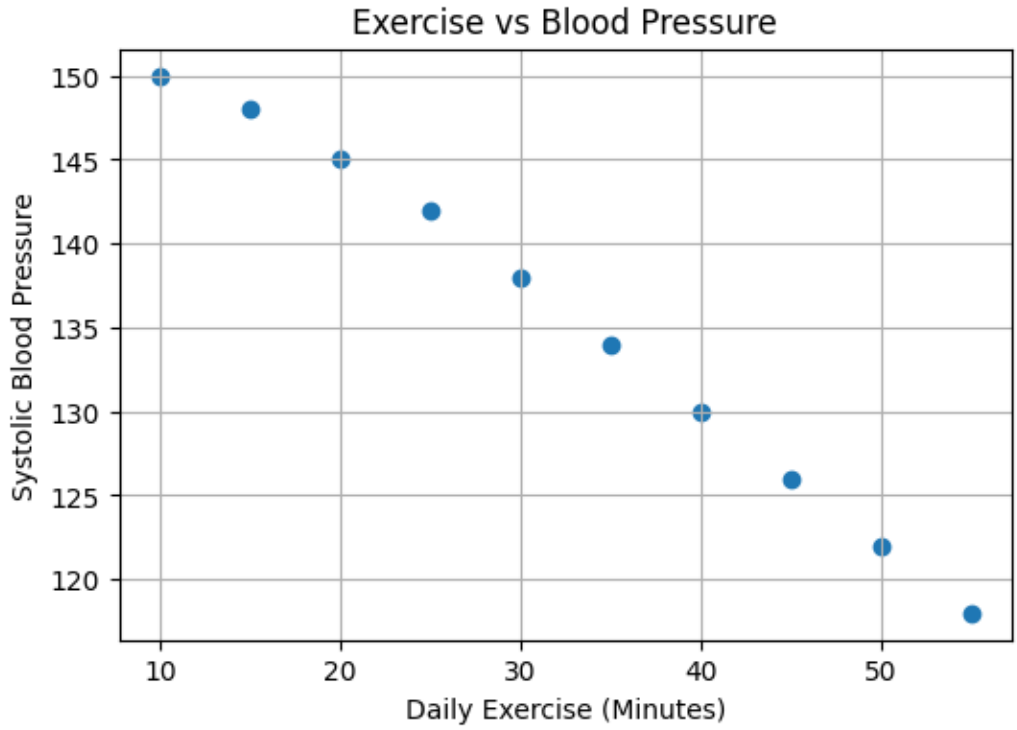
3. Calculate correlation coefficients.
4. Identify whether exercise or BMI is more strongly associated with blood pressure.
5. Discuss possible healthcare interventions.

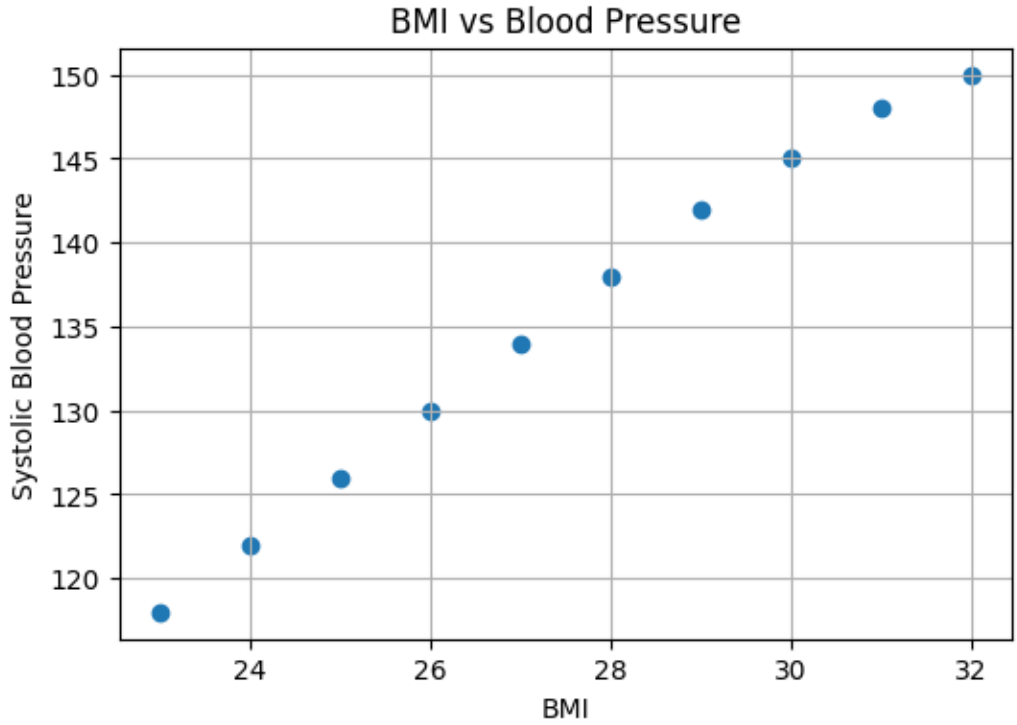
```
# ============================================
# Problem 1: Hospital Patient Health Monitoring
# ============================================

import pandas as pd
import matplotlib.pyplot as plt

# ----------------------------
# Create Dataset
# ----------------------------
data = {
   "Patient_ID": ["P001","P002","P003","P004","P005",
                  "P006","P007","P008","P009","P010"],

   "Daily_Exercise":[10,15,20,25,30,35,40,45,50,55],

   "BMI":[32,31,30,29,28,27,26,25,24,23],

   "Systolic_BP":[150,148,145,142,138,134,130,126,122,118]
}

df = pd.DataFrame(data)

print("="*60)
print("Hospital Patient Health Monitoring")
print("="*60)

print("\nDataset")
print(df)

# ================================================
# Task 1
# Mean and Standard Deviation of Blood Pressure
# ================================================
```

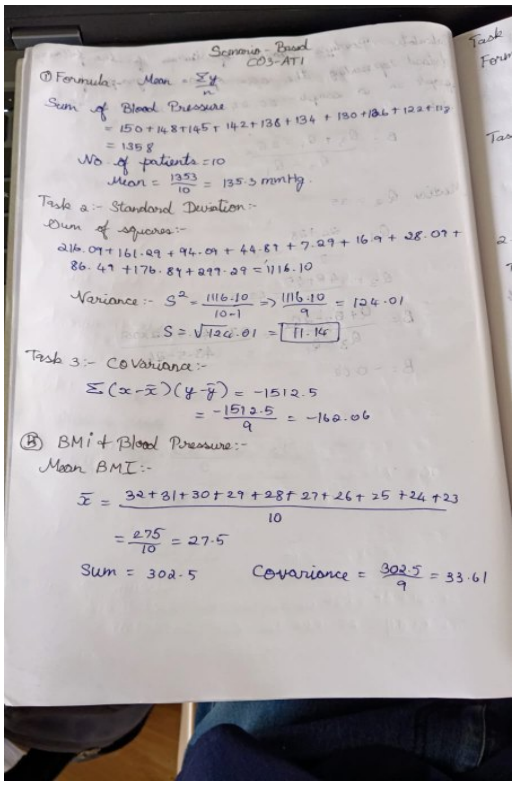
```
mean_bp = df["Systolic_BP"].mean()
std_bp = df["Systolic_BP"].std()

print("\nTask 1")
print("-------------------------")
print("Mean Blood Pressure =", round(mean_bp,2))
print("Standard Deviation =", round(std_bp,2))

# ================================================
# Task 2
# Covariance
# ================================================

cov_exercise = df["Daily_Exercise"].cov(df["Systolic_BP"])
cov_bmi = df["BMI"].cov(df["Systolic_BP"])

print("\nTask 2")
print("-------------------------")
print("Covariance (Exercise, BP) =", round(cov_exercise,2))
print("Covariance (BMI, BP) =", round(cov_bmi,2))

# ================================================
# Task 3
# Correlation
# ================================================

corr_exercise = df["Daily_Exercise"].corr(df["Systolic_BP"])
corr_bmi = df["BMI"].corr(df["Systolic_BP"])

print("\nTask 3")
print("-------------------------")
print("Correlation (Exercise, BP) =", round(corr_exercise,4))
print("Correlation (BMI, BP) =", round(corr_bmi,4))

# ================================================
# Task 4
# Stronger Association
```

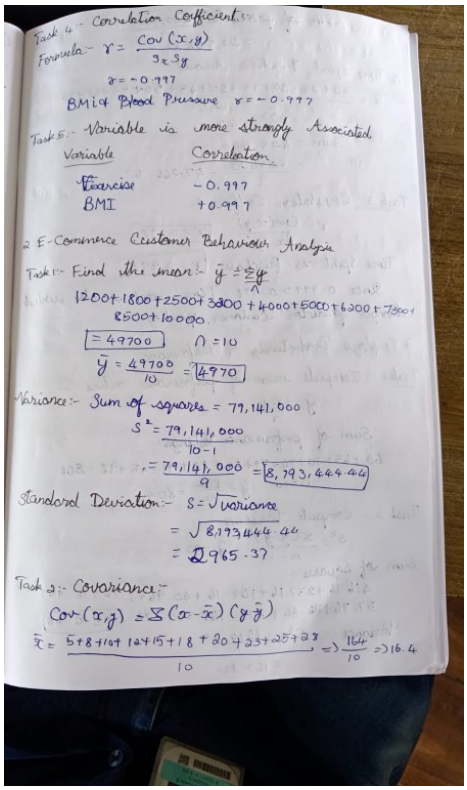
```
# ================================================

print("\nTask 4")
print("--------------------------")

if abs(corr_exercise) > abs(corr_bmi):
    print("Exercise is more strongly associated with Blood Pressure.")
elif abs(corr_exercise) < abs(corr_bmi):
    print("BMI is more strongly associated with Blood Pressure.")
else:
    print("Both Exercise and BMI have equal association with Blood Pressure.")

# ================================================
# Task 5
# Healthcare Interventions
# ================================================

print("\nTask 5")
print("--------------------------")

print("""
Possible Healthcare Interventions

1. Encourage daily physical exercise.
2. Promote healthy BMI through diet and exercise.
3. Conduct regular blood pressure monitoring.
4. Organize awareness programs on hypertension.
5. Recommend routine health checkups.
""")

# ================================================
# Task 6
# Scatter Plot : Exercise vs Blood Pressure
# ================================================

plt.figure(figsize=(6,4))
plt.scatter(df["Daily_Exercise"], df["Systolic_BP"])
plt.title("Exercise vs Blood Pressure")
```

```
plt.xlabel("Daily Exercise (Minutes)")
plt.ylabel("Systolic Blood Pressure")
plt.grid(True)
plt.show()

# ================================================
# Task 7
# Scatter Plot : BMI vs Blood Pressure
# ================================================

plt.figure(figsize=(6,4))
plt.scatter(df["BMI"], df["Systolic_BP"])
plt.title("BMI vs Blood Pressure")
plt.xlabel("BMI")
plt.ylabel("Systolic Blood Pressure")
plt.grid(True)
plt.show()
```









Problem 02: E-Commerce Customer Behaviour Analysis

Scenario

An e-commerce company wants to identify factors influencing customer spending.

Dataset

Customer_ID	Website_Visits	Time_Spent (Minutes)	Purchase_Amount (₹)

C001	5	20	1200
C002	8	25	1800
C003	10	30	2500
C004	12	35	3200
C005	15	40	4000
C006	18	45	5000
C007	20	50	6200
C008	23	55	7300
C009	25	60	8500
C010	28	70	10000

Tasks

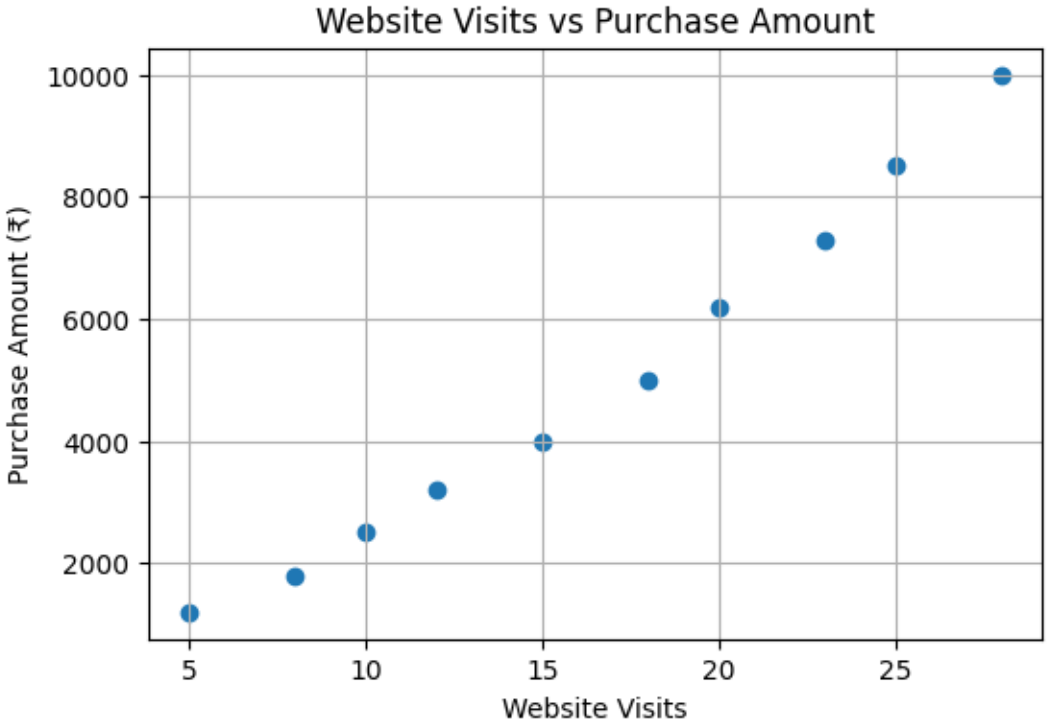
1. Calculate mean, variance, and standard deviation of Purchase Amount.
2. Find covariance between:
    - Website Visits and Purchase Amount
    - Time Spent and Purchase Amount
3. Calculate correlation coefficients.
4. Determine which customer behavior metric better predicts spending.
5. Suggest marketing strategies based on the findings.

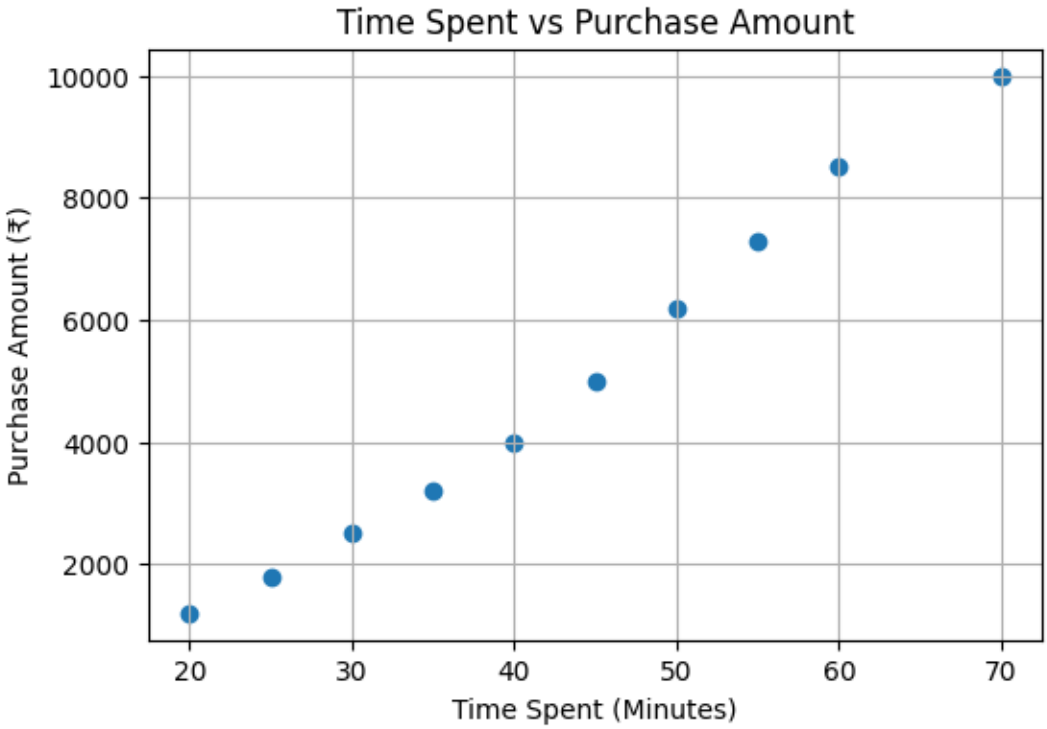
```
# ==========================================
# Problem 2: E-Commerce Customer Behaviour Analysis
# ==========================================

import pandas as pd
import matplotlib.pyplot as plt

# ----------------------------
# Create Dataset
# ----------------------------
data = {
   "Customer_ID": ["C001","C002","C003","C004","C005",
                   "C006","C007","C008","C009","C010"],

   "Website_Visits": [5,8,10,12,15,18,20,23,25,28],

   "Time_Spent": [20,25,30,35,40,45,50,55,60,70],

   "Purchase_Amount": [1200,1800,2500,3200,4000,
```

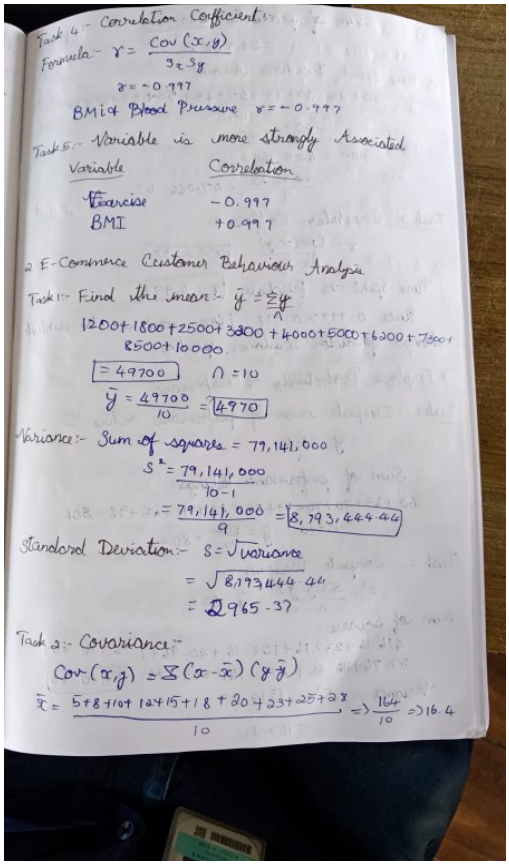
```
                5000,6200,7300,8500,10000]
}

df = pd.DataFrame(data)

print("="*60)
print("E-Commerce Customer Behaviour Analysis")
print("="*60)

print("\nDataset")
print(df)

# ================================================
# Task 1
# Mean, Variance and Standard Deviation
# ================================================

mean_purchase = df["Purchase_Amount"].mean()
variance_purchase = df["Purchase_Amount"].var()
std_purchase = df["Purchase_Amount"].std()

print("\nTask 1")
print("-"*40)
print("Mean Purchase Amount =", round(mean_purchase,2))
print("Variance =", round(variance_purchase,2))
print("Standard Deviation =", round(std_purchase,2))

# ================================================
# Task 2
# Covariance
# ================================================

cov_visits = df["Website_Visits"].cov(df["Purchase_Amount"])
cov_time = df["Time_Spent"].cov(df["Purchase_Amount"])

print("\nTask 2")
print("-"*40)
print("Covariance (Website Visits, Purchase Amount) =", round(cov_visits,2))
```

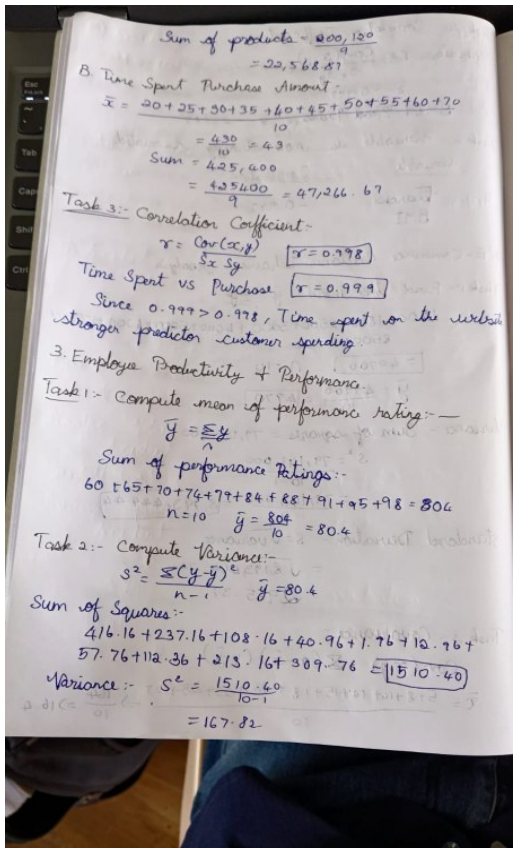
```
print("Covariance (Time Spent, Purchase Amount) =", round(cov_time,2))

# ================================================
# Task 3
# Correlation Coefficients
# ================================================

corr_visits = df["Website_Visits"].corr(df["Purchase_Amount"])
corr_time = df["Time_Spent"].corr(df["Purchase_Amount"])

print("\nTask 3")
print("-"*40)
print("Correlation (Website Visits, Purchase Amount) =", round(corr_visits,4))
print("Correlation (Time Spent, Purchase Amount) =", round(corr_time,4))

# ================================================
# Task 4
# Better Predictor
# ================================================

print("\nTask 4")
print("-"*40)

if abs(corr_visits) > abs(corr_time):
    print("Website Visits better predict customer spending.")
elif abs(corr_visits) < abs(corr_time):
    print("Time Spent better predicts customer spending.")
else:
    print("Both Website Visits and Time Spent are equally good predictors.")

# ================================================
# Task 5
# Marketing Strategies
# ================================================

print("\nTask 5")
print("-"*40)
```

```
print("""
Suggested Marketing Strategies

1. Improve website engagement with attractive content.
2. Recommend products based on customer browsing history.
3. Offer personalized discounts to frequent visitors.
4. Increase session duration using interactive features.
5. Use loyalty and reward programs to encourage repeat purchases.
""")

# ================================================
# Scatter Plot
# Website Visits vs Purchase Amount
# ================================================

plt.figure(figsize=(6,4))
plt.scatter(df["Website_Visits"], df["Purchase_Amount"], marker='o')
plt.title("Website Visits vs Purchase Amount")
plt.xlabel("Website Visits")
plt.ylabel("Purchase Amount (₹)")
plt.grid(True)
plt.show()

# ================================================
# Scatter Plot
# Time Spent vs Purchase Amount
# ================================================

plt.figure(figsize=(6,4))
plt.scatter(df["Time_Spent"], df["Purchase_Amount"], marker='o')
plt.title("Time Spent vs Purchase Amount")
plt.xlabel("Time Spent (Minutes)")
plt.ylabel("Purchase Amount (₹)")
plt.grid(True)
plt.show()
```


Website Visits vs Purchase Amount
Purchase Amount (₹)
10000
8000
6000
4000
2000
5
10
15
20
25
Website Visits


Time Spent vs Purchase Amount
Purchase Amount (₹)
10000
8000
6000
4000
2000
20
30
40
50
60
70
Time Spent (Minutes)





Problem 03: Employee Productivity and Performance

Scenario

A software company is investigating the relationship between employee productivity and performance ratings.

Dataset

Employee_ID	Training_Hours	Projects_Completed	Performance_Rating
E001	5	2	60
E002	8	3	65
E003	10	4	70
E004	12	5	74
E005	15	6	79
E006	18	7	84
E007	20	8	88
E008	22	9	91
E009	25	10	95
E010	28	11	98

Tasks

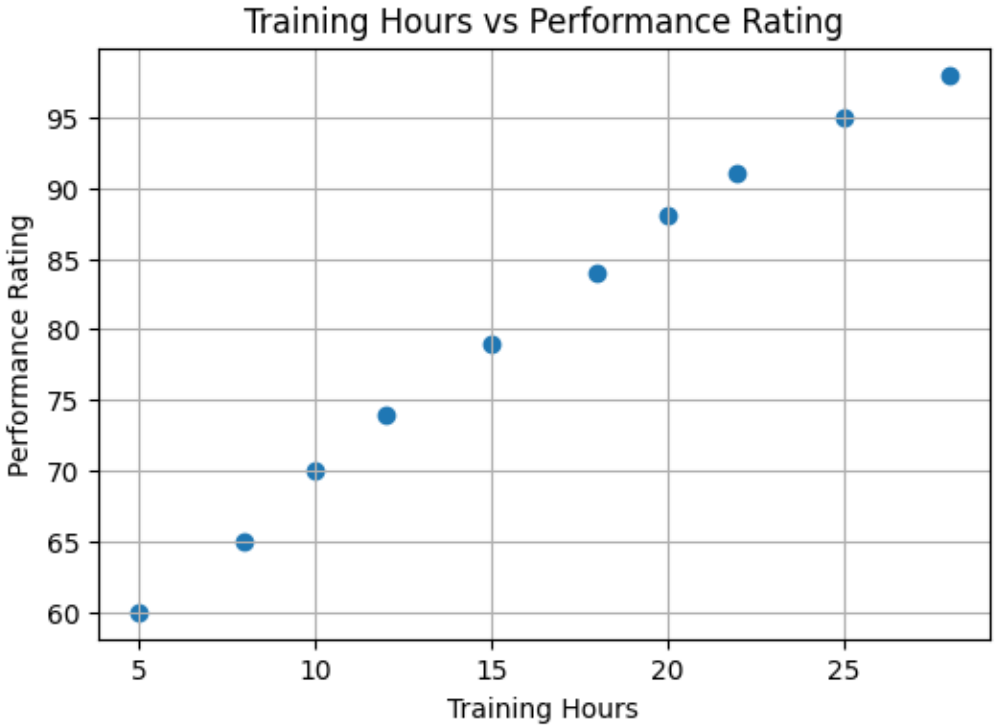
1. Compute the mean, variance, and standard deviation of Performance Ratings.
2. Calculate covariance between:
    - Training Hours and Performance Rating
    - Projects Completed and Performance Rating
3. Compute correlation coefficients.
4. Identify the strongest factor affecting employee performance.
5. Recommend HR policies to improve employee productivity.

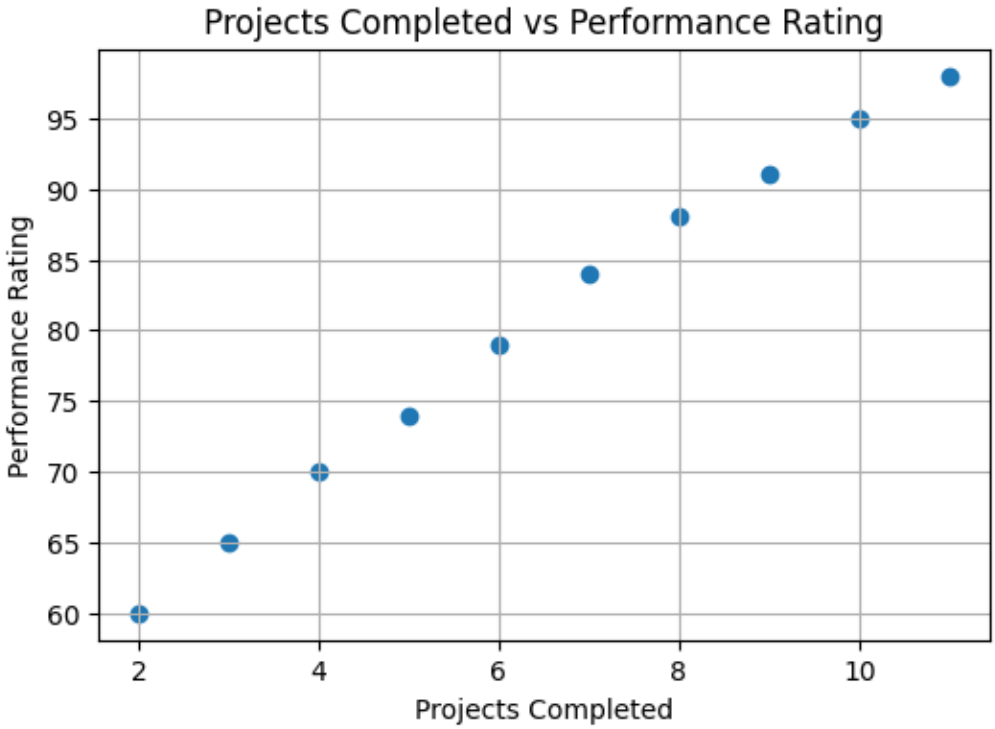
```
# ==========================================
# Problem 3: Employee Productivity and Performance
# ==========================================

import pandas as pd
import matplotlib.pyplot as plt

# ----------------------------
# Create Dataset
# ----------------------------
data = {
   "Employee_ID": ["E001","E002","E003","E004","E005",
                   "E006","E007","E008","E009","E010"],

   "Training_Hours": [5,8,10,12,15,18,20,22,25,28],

   "Projects_Completed": [2,3,4,5,6,7,8,9,10,11],

   "Performance_Rating": [60,65,70,74,79,84,88,91,95,98]
}

df = pd.DataFrame(data)

print("="*60)
print("Employee Productivity and Performance")
print("="*60)

print("\nDataset")
print(df)
```

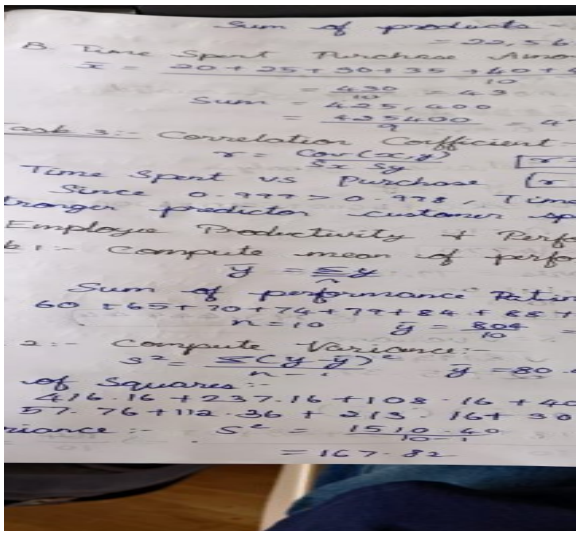
```
# ================================================
# Task 1
# Mean, Variance and Standard Deviation
# ================================================

mean_rating = df["Performance_Rating"].mean()
variance_rating = df["Performance_Rating"].var()
std_rating = df["Performance_Rating"].std()

print("\nTask 1")
print("-"*40)
print("Mean Performance Rating =", round(mean_rating,2))
print("Variance =", round(variance_rating,2))
print("Standard Deviation =", round(std_rating,2))

# ================================================
# Task 2
# Covariance
# ================================================

cov_training = df["Training_Hours"].cov(df["Performance_Rating"])
cov_projects = df["Projects_Completed"].cov(df["Performance_Rating"])

print("\nTask 2")
print("-"*40)
print("Covariance (Training Hours, Performance Rating) =", round(cov_training,2))
print("Covariance (Projects Completed, Performance Rating) =", round(cov_projects,2))

# ================================================
# Task 3
# Correlation Coefficients
# ================================================

corr_training = df["Training_Hours"].corr(df["Performance_Rating"])
corr_projects = df["Projects_Completed"].corr(df["Performance_Rating"])

print("\nTask 3")
print("-"*40)
```

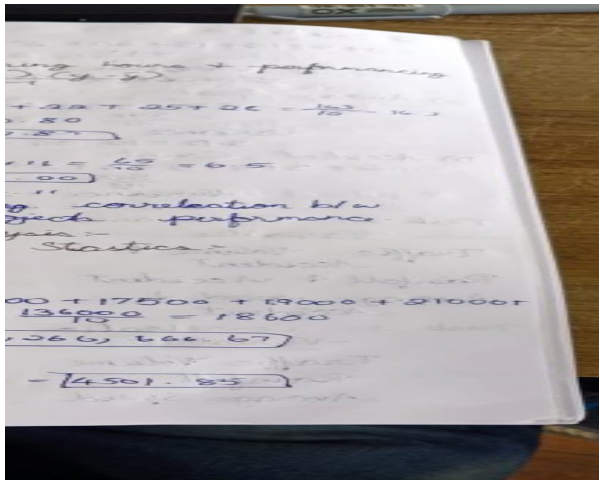
```
print("Correlation (Training Hours, Performance Rating) =", round(corr_training,4))
print("Correlation (Projects Completed, Performance Rating) =", round(corr_projects,4))

# ================================================
# Task 4
# Strongest Factor Affecting Performance
# ================================================

print("\nTask 4")
print("-"*40)

if abs(corr_training) > abs(corr_projects):
    print("Training Hours have a stronger effect on employee performance.")
elif abs(corr_training) < abs(corr_projects):
    print("Projects Completed have a stronger effect on employee performance.")
else:
    print("Training Hours and Projects Completed have an equal effect on employee
performance.")

# ================================================
# Task 5
# HR Recommendations
# ================================================

print("\nTask 5")
print("-"*40)

print("""
HR Recommendations

1. Conduct regular employee training programs.
2. Encourage employees to participate in more projects.
3. Provide performance-based incentives.
4. Organize skill development workshops.
5. Monitor employee performance periodically and provide constructive feedback.
""")

# ================================================
# Scatter Plot
```

```
# Training Hours vs Performance Rating
# ================================================

plt.figure(figsize=(6,4))
plt.scatter(df["Training_Hours"], df["Performance_Rating"], marker='o')
plt.title("Training Hours vs Performance Rating")
plt.xlabel("Training Hours")
plt.ylabel("Performance Rating")
plt.grid(True)
plt.show()

# ================================================
# Scatter Plot
# Projects Completed vs Performance Rating
# ================================================

plt.figure(figsize=(6,4))
plt.scatter(df["Projects_Completed"], df["Performance_Rating"], marker='o')
plt.title("Projects Completed vs Performance Rating")
plt.xlabel("Projects Completed")
plt.ylabel("Performance Rating")
plt.grid(True)
plt.show()
```


Training Hours vs Performance Rating
Performance Rating
Training Hours
60
65
70
75
80
85
90
95
5
10
15
20
25


Projects Completed vs Performance Rating
Performance Rating
Projects Completed
60
65
70
75
80
85
90
95
2
4
6
8
10





Problem 04 : Smart City Traffic Analysis

A city administration wants to analyze traffic congestion patterns.

Day	Traffic_Volume (Vehicles)	Rainfall (mm)	Average_Speed (km/h)	Accident_Count
D1	12000	0	55	2
D2	13500	5	50	3
D3	14500	10	48	4
D4	16000	15	45	5
D5	17500	20	42	6
D6	19000	25	38	8
D7	21000	30	35	10
D8	22500	35	32	12
D9	24000	40	28	14
D10	26000	45	25	16

Tasks

1. Calculate descriptive statistics (Mean, Variance, Standard Deviation) for all variables.
2. Compute covariance matrices.
3. Compute correlation matrices.
4. Identify positive and negative relationships among variables.
5. Determine which factor is most associated with accident occurrence.
6. Provide data-driven recommendations to reduce traffic accidents.
7. Visualize relationships using scatter plots and interpret the results.

==========

Problem 4: Smart City Traffic Analysis

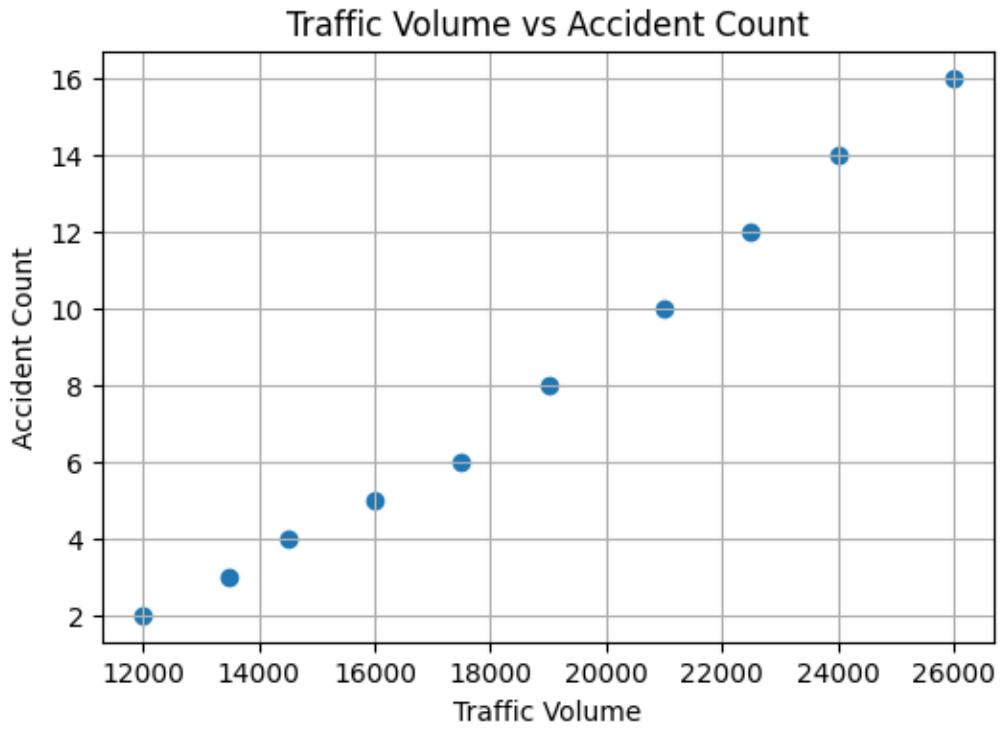
```
# ==========================================

import pandas as pd
import matplotlib.pyplot as plt

# ----------------------------
# Create Dataset
# ----------------------------
data = {
   "Day": ["D1","D2","D3","D4","D5",
         "D6","D7","D8","D9","D10"],

   "Traffic_Volume": [12000,13500,14500,16000,17500,
                  19000,21000,22500,24000,26000],

   "Rainfall": [0,5,10,15,20,25,30,35,40,45],

   "Average_Speed": [55,50,48,45,42,38,35,32,28,25],

   "Accident_Count": [2,3,4,5,6,8,10,12,14,16]
}

df = pd.DataFrame(data)

print("="*70)
print("Problem 4: Smart City Traffic Analysis")
print("="*70)

print("\nDataset")
print(df)

# ================================================
# Task 1
# Mean, Variance and Standard Deviation
# ================================================

print("\nTask 1")
print("-"*50)
```

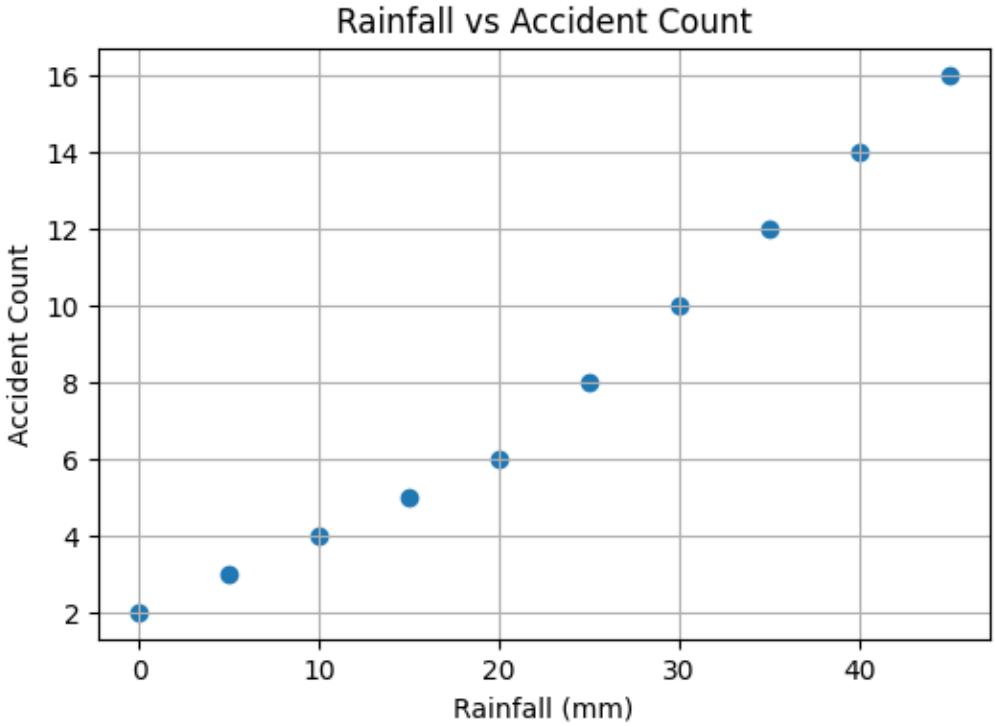
```
stats_df = pd.DataFrame({
    "Mean": df.iloc[:,1:].mean(),
    "Variance": df.iloc[:,1:].var(),
    "Standard Deviation": df.iloc[:,1:].std()
})

print(stats_df)

# ==================================================
# Task 2
# Covariance Matrix
# ==================================================

print("\nTask 2")
print("-"*50)

cov_matrix = df.iloc[:,1:].cov()

print("Covariance Matrix:")
print(cov_matrix)

# ==================================================
# Task 3
# Correlation Matrix
# ==================================================

print("\nTask 3")
print("-"*50)

corr_matrix = df.iloc[:,1:].corr()

print("Correlation Matrix:")
print(corr_matrix)

# ==================================================
# Task 4
# Positive and Negative Relationships
```

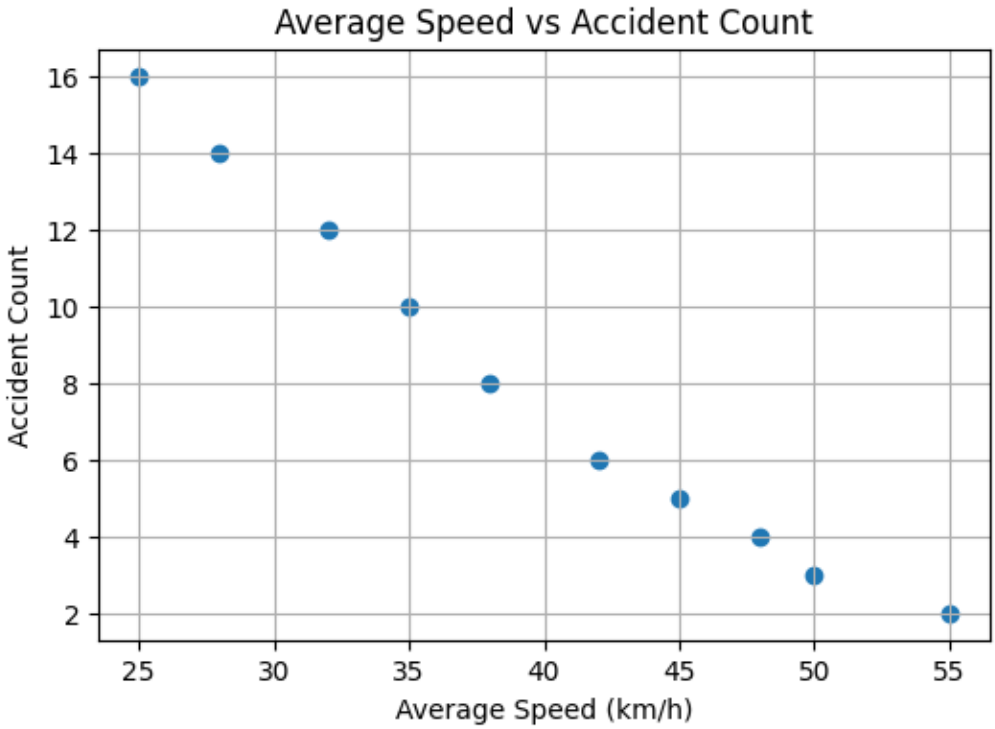
```
# ================================================

print("\nTask 4")
print("-"*50)

print("Positive Relationships:")
print("- Traffic Volume and Accident Count")
print("- Rainfall and Accident Count")
print("- Traffic Volume and Rainfall")

print("\nNegative Relationships:")
print("- Average Speed and Accident Count")
print("- Average Speed and Traffic Volume")
print("- Average Speed and Rainfall")

# ================================================
# Task 5
# Factor Most Associated with Accidents
# ================================================

print("\nTask 5")
print("-"*50)

accident_corr = corr_matrix["Accident_Count"].drop("Accident_Count")

print("Correlation with Accident Count:")
print(accident_corr)

strongest = accident_corr.abs().idxmax()

print("\nMost Associated Factor:", strongest)

# ================================================
# Task 6
# Recommendations
# ================================================

print("\nTask 6")
```

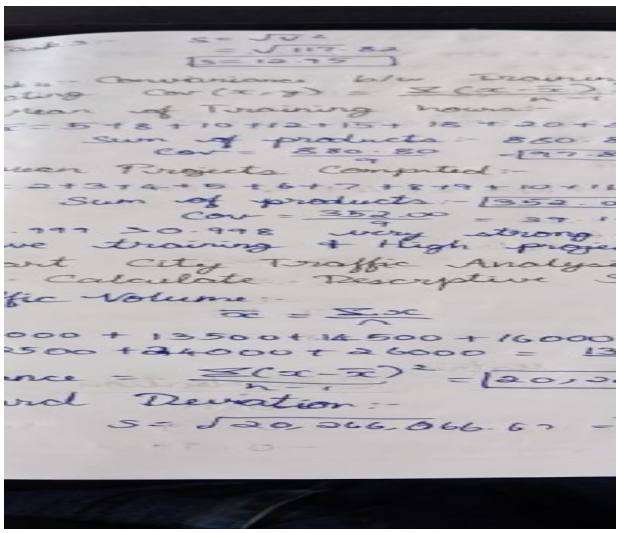
```
print("-"*50)

print("""
Recommendations

1. Improve traffic management during peak hours.
2. Install speed monitoring systems.
3. Enhance road drainage to reduce rain-related accidents.
4. Increase traffic police deployment in congested areas.
5. Promote road safety awareness campaigns.
6. Optimize traffic signals using smart city technologies.
""")

# ================================================
# Task 7
# Scatter Plots
# ================================================

# Traffic Volume vs Accident Count
plt.figure(figsize=(6,4))
plt.scatter(df["Traffic_Volume"], df["Accident_Count"])
plt.title("Traffic Volume vs Accident Count")
plt.xlabel("Traffic Volume")
plt.ylabel("Accident Count")
plt.grid(True)
plt.show()

# Rainfall vs Accident Count
plt.figure(figsize=(6,4))
plt.scatter(df["Rainfall"], df["Accident_Count"])
plt.title("Rainfall vs Accident Count")
plt.xlabel("Rainfall (mm)")
plt.ylabel("Accident Count")
plt.grid(True)
plt.show()

# Average Speed vs Accident Count
plt.figure(figsize=(6,4))
```

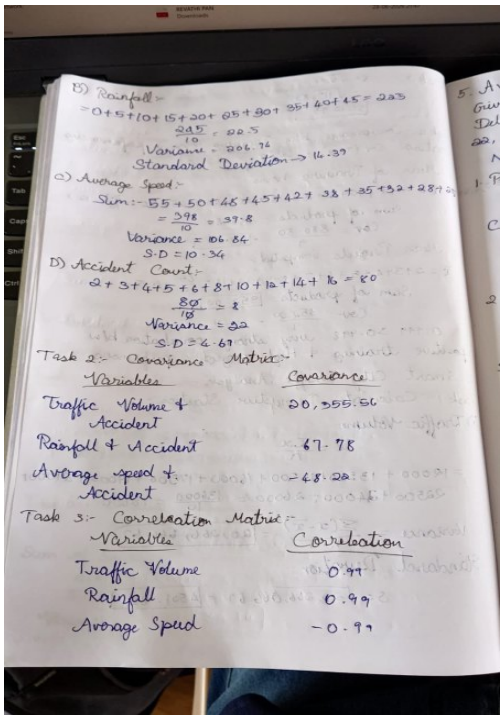
```
plt.scatter(df["Average_Speed"], df["Accident_Count"])
plt.title("Average Speed vs Accident Count")
plt.xlabel("Average Speed (km/h)")
plt.ylabel("Accident Count")
plt.grid(True)
plt.show()

# ================================================
# Interpretation
# ================================================

print("\nInterpretation")
print("-"*50)

print("""
1. Traffic Volume and Rainfall show a strong positive relationship with Accident Count.
2. Average Speed has a strong negative relationship with Accident Count.
3. As traffic volume and rainfall increase, accidents tend to increase.
4. As average speed decreases due to congestion, accident occurrence increases.
5. Smart traffic management and better road infrastructure can reduce accidents.
""")
```




Rainfall vs Accident Count
Accident Count
Rainfall (mm)
0
10
20
30
40
2
4
6
8
10
12
14
16


Average Speed vs Accident Count
Accident Count
Average Speed (km/h)
25
30
35
40
45
50
55
2
4
6
8
10
12
14
16





Problem 5: Average Delivery Time

A food delivery company records the delivery times (minutes) for 15 orders:
22, 25, 18, 30, 27, 24, 21, 19, 26, 23, 28, 20, 24, 25, 22

Tasks:

1. Find the point estimate of the population mean delivery time.
2. Find the point estimate of the population variance.
3. Interpret the estimates.

```
# ==========================================
# Problem 5: Average Delivery Time
# ==========================================

import pandas as pd

# ----------------------------
# Create Dataset
# ----------------------------
delivery_time = [22, 25, 18, 30, 27, 24, 21, 19, 26, 23, 28, 20, 24, 25, 22]

# Convert into DataFrame
df = pd.DataFrame({"Delivery_Time": delivery_time})
```

```
print("="*60)
print("Problem 5: Average Delivery Time")
print("="*60)

print("\nDataset")
print(df)

# ================================================
# Task 1
# Point Estimate of Population Mean
# ================================================

mean_delivery = df["Delivery_Time"].mean()

print("\nTask 1")
print("-"*40)
print("Point Estimate of Population Mean =", round(mean_delivery,2), "minutes")

# ================================================
# Task 2
# Point Estimate of Population Variance
# ================================================

variance_delivery = df["Delivery_Time"].var()

print("\nTask 2")
print("-"*40)
print("Point Estimate of Population Variance =", round(variance_delivery,2))

# ================================================
# Task 3
# Interpretation
```

```
# ================================================

print("\nTask 3")
print("-"*40)

print(f"""
Interpretation

1. The estimated average delivery time is {mean_delivery:.2f} minutes.
2. The estimated variance is {variance_delivery:.2f}, indicating the spread of delivery times.
3. These values are point estimates because they are calculated from the sample and used to estimate the corresponding population parameters.
""")

============================================================
Problem 5: Average Delivery Time
============================================================

Dataset
    Delivery_Time
0           22
1           25
2           18
3           30
4           27
5           24
6           21
7           19
8           26
9           23
10          28
11          20
12          24
```

13 25

14 22

Task 1

Point Estimate of Population Mean = 23.6 minutes

Task 2

Point Estimate of Population Variance = 11.4

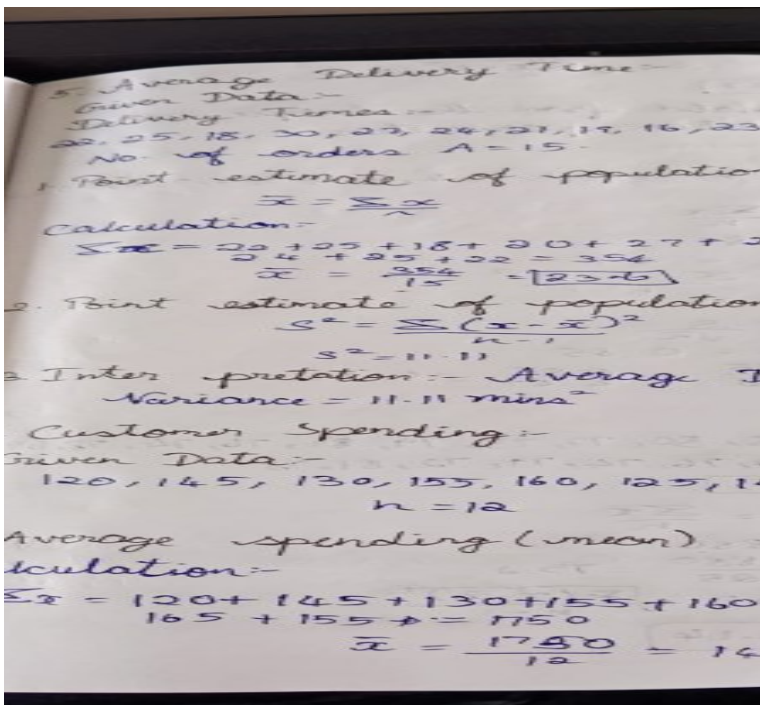
Task 3

Interpretation

1. The estimated average delivery time is 23.60 minutes.

2. The estimated variance is 11.40, indicating the spread of delivery times.

3. These values are point estimates because they are calculated from the sample and used to estimate the corresponding population parameters.



Problem 6: Customer Spending

A sample of 12 customers spent the following amounts ($):

120, 145, 130, 155, 160, 125, 140, 135, 150, 170, 165, 155

Tasks:

1. Estimate the average spending.
2. Estimate the standard deviation.
3. Explain why these are point estimates.

```
# ============================================
# Problem 6: Customer Spending
# ============================================

import pandas as pd

# ---------------------------
# Create Dataset
# ---------------------------
customer_spending = [120, 145, 130, 155, 160, 125, 140, 135, 150, 170, 165, 155]

# Convert into DataFrame
df = pd.DataFrame({"Customer_Spending": customer_spending})

print("="*60)
print("Problem 6: Customer Spending")
print("="*60)

print("\nDataset")
print(df)

# ================================================
# Task 1
# Estimate the Average Spending
# ================================================
```

```
average_spending = df["Customer_Spending"].mean()

print("\nTask 1")
print("-"*40)
print("Estimated Average Spending = $", round(average_spending, 2))

# ==================================================
# Task 2
# Estimate the Standard Deviation
# ==================================================

std_spending = df["Customer_Spending"].std()

print("\nTask 2")
print("-"*40)
print("Estimated Standard Deviation =", round(std_spending, 2))

# ==================================================
# Task 3
# Interpretation
# ==================================================

print("\nTask 3")
print("-"*40)

print(f"""
Interpretation

1. The estimated average customer spending is ${average_spending:.2f}.
2. The estimated standard deviation is {std_spending:.2f}, which indicates how
much the spending values vary around the average.
3. These are called point estimates because they are single numerical values
calculated from the sample and are used to estimate the corresponding population
parameters.
""")
```

==========

Problem 6: Customer Spending

==========

Dataset

	Customer_Spending
0	120
1	145

2	130
3	155
4	160
5	125
6	140
7	135
8	150
9	170
10	165
11	155

Task 1

Estimated Average Spending = $ 145.83

Task 2

Estimated Standard Deviation = 16.07

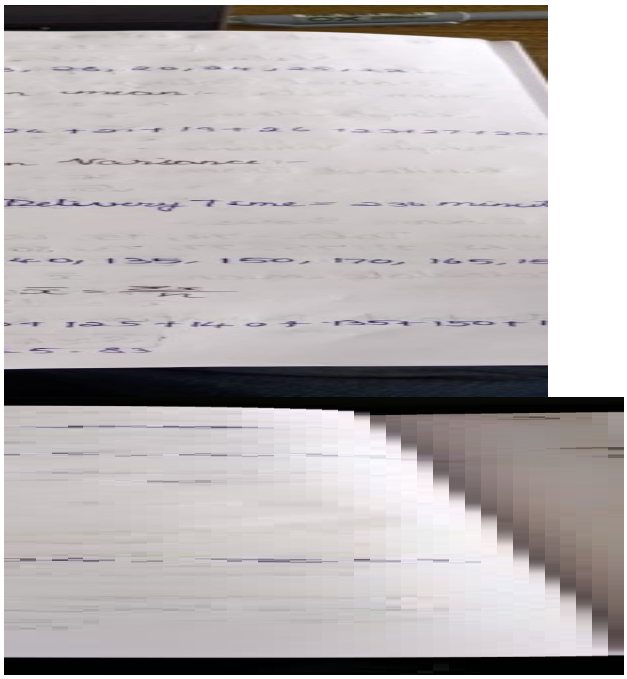
Task 3

Interpretation

1. The estimated average customer spending is $145.83.

2. The estimated standard deviation is 16.07, which indicates how much the spending values vary around the average.

3. These are called point estimates because they are single numerical values calculated from the sample and are used to estimate the corresponding population parameters.



Problem 7: Product Ratings

A company collects ratings from 20 customers:

4, 5, 3, 4, 5, 4, 3, 5, 4, 4, 5, 3, 4, 5, 4, 3, 5, 4, 5, 4

Tasks:

1. Compute the sample mean.
2. Compute the sample variance.
3. Compute the standard error of the mean.
4. Explain what the standard error indicates.

```
# ============================================
# Problem 7: Product Ratings
# ============================================

import pandas as pd
import numpy as np

# ----------------------------
# Create Dataset
# ----------------------------
ratings = [4, 5, 3, 4, 5, 4, 3, 5, 4, 4,
           5, 3, 4, 5, 4, 3, 5, 4, 5, 4]
```

```
# Convert into DataFrame
df = pd.DataFrame({"Product_Ratings": ratings})

print("="*60)
print("Problem 7: Product Ratings")
print("="*60)

print("\nDataset")
print(df)

# ================================================
# Task 1
# Compute the Sample Mean
# ================================================

sample_mean = df["Product_Ratings"].mean()

print("\nTask 1")
print("-"*40)
print("Sample Mean =", round(sample_mean, 2))

# ================================================
# Task 2
# Compute the Sample Variance
# ================================================

sample_variance = df["Product_Ratings"].var()

print("\nTask 2")
print("-"*40)
print("Sample Variance =", round(sample_variance, 4))
```

```
# ================================================
# Task 3
# Compute the Standard Error of the Mean
# ================================================

sample_std = df["Product_Ratings"].std()
sample_size = len(df)

standard_error = sample_std / np.sqrt(sample_size)

print("\nTask 3")
print("-"*40)
print("Standard Error =", round(standard_error, 4))

# ================================================
# Task 4
# Interpretation
# ================================================

print("\nTask 4")
print("-"*40)

print(f"""
Interpretation

1. The sample mean rating is {sample_mean:.2f}.
2. The sample variance is {sample_variance:.4f}.
3. The standard error of the mean is {standard_error:.4f}.
4. A smaller standard error indicates that the sample mean is likely to be a reliable estimate of the population mean.
""")

============================================================
```

Problem 7: Product Ratings

==========

Dataset

	Product_Ratings
0	4
1	5
2	3
3	4
4	5
5	4
6	3
7	5
8	4
9	4
10	5
11	3
12	4
13	5
14	4
15	3
16	5
17	4
18	5
19	4

Task 1

Sample Mean = 4.15

Task 2

Sample Variance = 0.5553

Task 3

Standard Error = 0.1666

Task 4

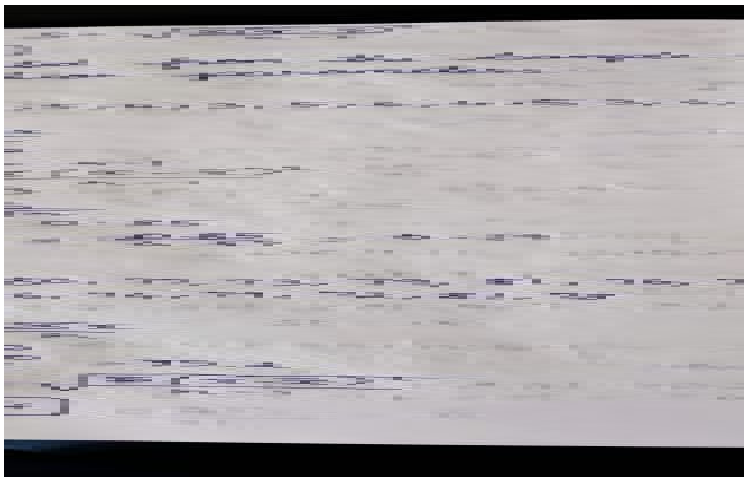
Interpretation

1. The sample mean rating is 4.15.

2. The sample variance is 0.5553.

3. The standard error of the mean is 0.1666.

4. A smaller standard error indicates that the sample mean is likely to be a reliable estimate of the population mean.



Problem 8: Exam Scores

Scores of 25 students:

72, 75, 68, 80, 77, 73, 79, 81, 76, 74, 78, 69, 70, 82, 71, 75, 77, 73, 80, 76, 74, 79, 72, 81, 78

Tasks:

1. Calculate mean and standard deviation.
2. Calculate standard error.
3. Compare standard deviation and standard error.

==========

Problem 8: Exam Scores

==========

```
import pandas as pd
import numpy as np

# ---------------------------
# Create Dataset
# ---------------------------
scores = [72, 75, 68, 80, 77, 73, 79, 81, 76, 74,
          78, 69, 70, 82, 71, 75, 77, 73, 80, 76,
          74, 79, 72, 81, 78]

# Convert into DataFrame
df = pd.DataFrame({"Exam_Scores": scores})

print("="*60)
print("Problem 8: Exam Scores")
print("="*60)

print("\nDataset")
print(df)

# ================================================
# Task 1
# Calculate Mean and Standard Deviation
# ================================================

mean_score = df["Exam_Scores"].mean()
std_score = df["Exam_Scores"].std()

print("\nTask 1")
print("-"*40)
print("Mean Score =", round(mean_score, 2))
print("Standard Deviation =", round(std_score, 2))
```

```
# ================================================
# Task 2
# Calculate Standard Error
# ================================================

sample_size = len(df)
standard_error = std_score / np.sqrt(sample_size)

print("\nTask 2")
print("-"*40)
print("Standard Error =", round(standard_error, 4))

# ================================================
# Task 3
# Compare Standard Deviation and Standard Error
# ================================================

print("\nTask 3")
print("-"*40)

print(f"""
Comparison

Mean Score          : {mean_score:.2f}
Standard Deviation  : {std_score:.2f}
Standard Error      : {standard_error:.4f}

Interpretation

1. The Standard Deviation measures the spread of individual exam scores around the mean.
2. The Standard Error measures the accuracy of the sample mean as an estimate of the population mean.
```

3. The Standard Error is smaller than the Standard Deviation because it is calculated by dividing the Standard Deviation by the square root of the sample size.

4. As the sample size increases, the Standard Error decreases, indicating a more precise estimate of the population mean.

""")

==========

Problem 8: Exam Scores

==========

Dataset

	Exam_Scores
0	72
1	75
2	68
3	80
4	77
5	73
6	79
7	81
8	76
9	74
10	78
11	69
12	70
13	82
14	71
15	75
16	77
17	73
18	80
19	76
20	74
21	79

22 72

23 81

24 78

Task 1

Mean Score = 75.6

Standard Deviation = 3.96

Task 2

Standard Error = 0.7916

Task 3

Comparison

Mean Score : 75.60

Standard Deviation : 3.96

Standard Error : 0.7916

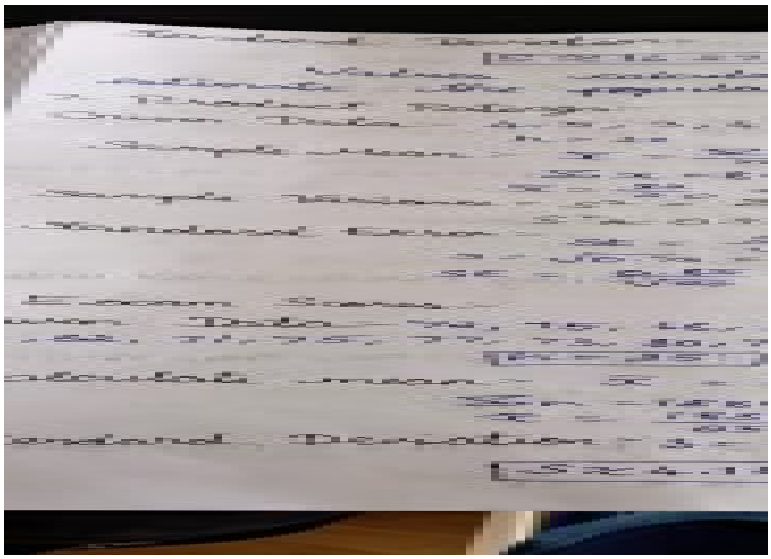
Interpretation

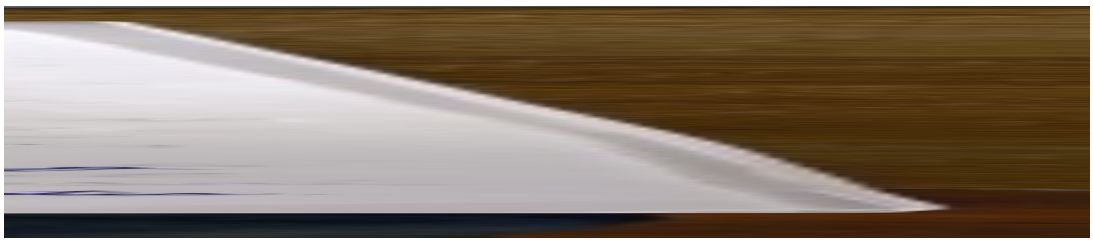
1. The Standard Deviation measures the spread of individual exam scores around the mean.

2. The Standard Error measures the accuracy of the sample mean as an estimate of the population mean.

3. The Standard Error is smaller than the Standard Deviation because it is calculated by dividing the Standard Deviation by the square root of the sample size.

4. As the sample size increases, the Standard Error decreases, indicating a more precise estimate of the population mean.





Problem 9: Manufacturing Weights

A sample of 50 packets has:

Mean weight = 498 g

Standard deviation = 12 g

Tasks:

1. Construct a 95% confidence interval for the population mean weight.
2. Interpret the interval.
3. Determine whether the target weight of 500 g is plausible.

```
# ============================================
# Problem 9: Manufacturing Weights
# ============================================

import numpy as np
from scipy.stats import t

print("="*60)
print("Problem 9: Manufacturing Weights")
print("="*60)

# ----------------------------
# Given Data
# ----------------------------
sample_mean = 498         # grams
```

```
sample_sd = 12          # grams
sample_size = 50
confidence_level = 0.95

print("\nGiven Data")
print("-"*40)
print("Sample Mean =", sample_mean)
print("Sample Standard Deviation =", sample_sd)
print("Sample Size =", sample_size)

# ================================================
# Task 1
# Construct a 95% Confidence Interval
# ================================================

alpha = 1 - confidence_level

# Critical t-value
t_critical = t.ppf(1 - alpha/2, sample_size - 1)

# Margin of Error
margin_error = t_critical * (sample_sd / np.sqrt(sample_size))

lower_limit = sample_mean - margin_error
upper_limit = sample_mean + margin_error

print("\nTask 1")
print("-"*40)
print("95% Confidence Interval")
print(f"({lower_limit:.2f}, {upper_limit:.2f}) grams")

# ================================================
# Task 2
# Interpretation
# ================================================

print("\nTask 2")
print("-"*40)

print(f"""
Interpretation

The 95% confidence interval for the population mean weight is

({lower_limit:.2f}, {upper_limit:.2f}) grams.

This means we are 95% confident that the true population
mean packet weight lies within this interval.
```

```
""")

# ================================================
# Task 3
# Check Whether Target Weight is Plausible
# ================================================

print("\nTask 3")
print("-"*40)

target_weight = 500

if lower_limit <= target_weight <= upper_limit:
    print(f"The target weight ({target_weight} g) lies within the confidence interval.")
    print("Therefore, the target weight is plausible.")
else:
    print(f"The target weight ({target_weight} g) does NOT lie within the confidence
interval.")
    print("Therefore, the target weight is not plausible.")
```

```
============================================================
Problem 9: Manufacturing Weights
============================================================


Given Data
----------------------------------------
Sample Mean = 498
Sample Standard Deviation = 12
Sample Size = 50


Task 1
----------------------------------------
95% Confidence Interval
(494.59, 501.41) grams


Task 2
----------------------------------------


Interpretation
```

The 95% confidence interval for the population mean weight is

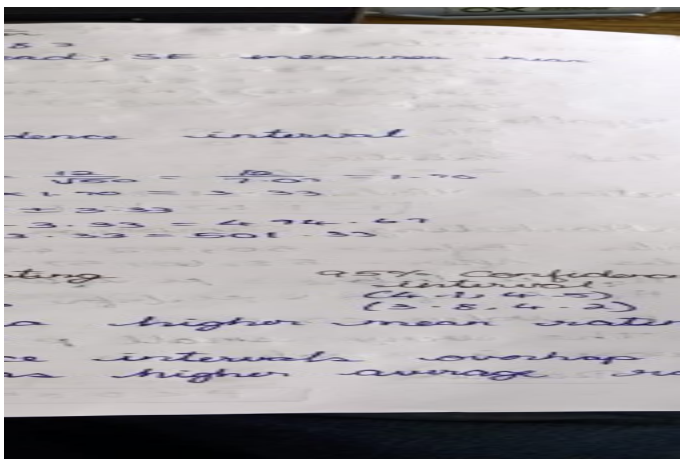
(494.59, 501.41) grams.

This means we are 95% confident that the true population mean packet weight lies within this interval.

Task 3

The target weight (500 g) lies within the confidence interval.

Therefore, the target weight is plausible.



Problem 10: Customer Satisfaction

Two branches report:

Branch	Mean Rating	95% CI
A	4.3	(4.1, 4.5)
B	4.0	(3.8,

4.2)

Tasks:

1. Compare the confidence intervals.
2. Can you conclude Branch A performs better?
3. Explain your reasoning.

```
# ============================================
# Problem 10: Customer Satisfaction
# ============================================

print("="*60)
print("Problem 10: Customer Satisfaction")
print("="*60)

# ----------------------------
# Given Data
# ----------------------------
branch_A_mean = 4.3
branch_A_CI = (4.1, 4.5)

branch_B_mean = 4.0
branch_B_CI = (3.8, 4.2)

print("\nBranch Details")
print("-"*40)

print(f"Branch A Mean Rating : {branch_A_mean}")
print(f"95% Confidence Interval : {branch_A_CI}")

print()

print(f"Branch B Mean Rating : {branch_B_mean}")
print(f"95% Confidence Interval : {branch_B_CI}")
```

```
# ================================================
# Task 1
# Compare the Confidence Intervals
# ================================================

print("\nTask 1")
print("-"*40)

print("Branch A CI :", branch_A_CI)
print("Branch B CI :", branch_B_CI)

if (branch_A_CI[1] < branch_B_CI[0]) or (branch_B_CI[1] < branch_A_CI[0]):
    print("\nThe confidence intervals do NOT overlap.")
else:
    print("\nThe confidence intervals overlap.")

# ================================================
# Task 2
# Can we conclude Branch A performs better?
# ================================================

print("\nTask 2")
print("-"*40)

if (branch_A_CI[0] > branch_B_CI[1]):
    print("Yes, Branch A performs significantly better.")
else:
    print("No, we cannot conclude that Branch A performs significantly better.")

# ================================================
# Task 3
# Explanation
# ================================================
```

```
print("\nTask 3")
print("-"*40)

print("""
Explanation

1. Branch A has a higher average rating (4.3) than Branch B (4.0).

2. The 95% confidence interval for Branch A is:
   (4.1, 4.5)

3. The 95% confidence interval for Branch B is:
   (3.8, 4.2)

4. Since the confidence intervals overlap (between 4.1 and 4.2),
   the difference may not be statistically significant.

5. Therefore, based only on these confidence intervals,
   we cannot confidently conclude that Branch A performs
   better than Branch B.
""")

============================================================
Problem 10: Customer Satisfaction
============================================================

Branch Details
----------------------------------
Branch A Mean Rating : 4.3
95% Confidence Interval : (4.1, 4.5)

Branch B Mean Rating : 4.0
```

95% Confidence Interval : (3.8, 4.2)

Task 1

Branch A CI : (4.1, 4.5)

Branch B CI : (3.8, 4.2)

The confidence intervals overlap.

Task 2

No, we cannot conclude that Branch A performs significantly better.

Task 3

Explanation

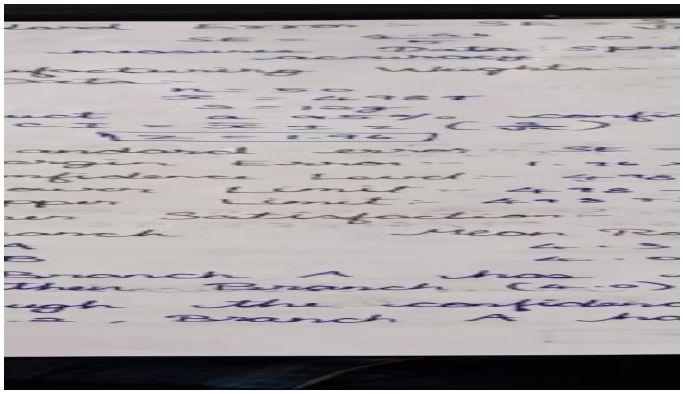
1. Branch A has a higher average rating (4.3) than Branch B (4.0).

2. The 95% confidence interval for Branch A is:
   (4.1, 4.5)

3. The 95% confidence interval for Branch B is:
   (3.8, 4.2)

4. Since the confidence intervals overlap (between 4.1 and 4.2),
   the difference may not be statistically significant.

5. Therefore, based only on these confidence intervals,
   we cannot confidently conclude that Branch A performs
   better than Branch B.



Problem 11: Average Battery Life

A battery manufacturer claims the average battery life is 10 hours.

A sample of 36 batteries gives:

Mean = 9.5 hours

Standard deviation = 1.8 hours

Tasks:

1. Formulate null and alternative hypotheses.
2. Conduct a hypothesis test at α = 0.05.
3. State your conclusion.

```
# ============================================
# Problem 11: Average Battery Life
# ============================================

import numpy as np
from scipy.stats import t

print("="*60)
print("Problem 11: Average Battery Life")
print("="*60)

# ----------------------------
# Given Data
# ----------------------------
population_mean = 10      # Claimed battery life (hours)
```

```
sample_mean = 9.5        # Sample mean
sample_sd = 1.8          # Sample standard deviation
sample_size = 36
alpha = 0.05

print("\nGiven Data")
print("-"*40)
print("Claimed Mean Battery Life =", population_mean, "hours")
print("Sample Mean =", sample_mean, "hours")
print("Sample Standard Deviation =", sample_sd)
print("Sample Size =", sample_size)

# ================================================
# Task 1
# Formulate Hypotheses
# ================================================

print("\nTask 1")
print("-"*40)

print("Null Hypothesis (H₀): μ = 10 hours")
print("Alternative Hypothesis (H₁): μ ≠ 10 hours")

# ================================================
# Task 2
# Conduct One-Sample t-Test
# ================================================

# Calculate t-statistic
t_statistic = (sample_mean - population_mean) / (sample_sd / np.sqrt(sample_size))

# Degrees of freedom
df = sample_size - 1
```

```
# Two-tailed p-value
p_value = 2 * (1 - t.cdf(abs(t_statistic), df))

print("\nTask 2")
print("-"*40)

print("t-statistic =", round(t_statistic, 4))
print("Degrees of Freedom =", df)
print("p-value =", round(p_value, 6))

# ================================================
# Task 3
# Conclusion
# ================================================

print("\nTask 3")
print("-"*40)

if p_value < alpha:
    print("Decision: Reject the Null Hypothesis (H₀)")
    print("Conclusion: There is sufficient evidence that the average battery life is significantly different from 10 hours.")
else:
    print("Decision: Fail to Reject the Null Hypothesis (H₀)")
    print("Conclusion: There is insufficient evidence to conclude that the average battery life differs from 10 hours.")

print("\nLevel of Significance (α) =", alpha)

============================================================
Problem 11: Average Battery Life
============================================================
```

Given Data

Claimed Mean Battery Life = 10 hours

Sample Mean = 9.5 hours

Sample Standard Deviation = 1.8

Sample Size = 36

Task 1

Null Hypothesis (H_0): $\mu = 10$ hours

Alternative Hypothesis (H_1): $\mu \neq 10$ hours

Task 2

---

t-statistic = -1.6667

Degrees of Freedom = 35

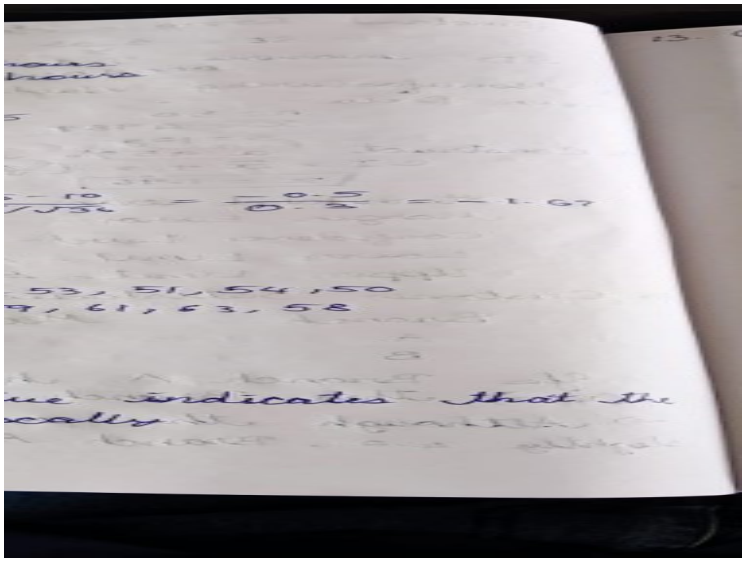
p-value = 0.104507

Task 3

Decision: Fail to Reject the Null Hypothesis (H_0)

Conclusion: There is insufficient evidence to conclude that the average battery life differs from 10 hours.

Level of Significance (α) = 0.05



Problem 12: Marketing Campaign

A company believes a new campaign increases sales.

Sales before campaign:

50, 52, 49, 55, 53, 51, 54, 50

Sales after campaign:

58, 60, 57, 62, 59, 61, 63, 58

Tasks:

1. Conduct a hypothesis test.
2. Calculate the p-value.
3. Interpret the p-value.
4. Make a decision at α = 0.05.

```
# ==========================================
# Problem 12: Marketing Campaign
# ==========================================

import numpy as np
from scipy.stats import ttest_ind

print("="*60)
print("Problem 12: Marketing Campaign")
print("="*60)
```

```
# ---------------------------
# Given Data
# ---------------------------
sales_before = [50, 52, 49, 55, 53, 51, 54, 50]
sales_after = [58, 60, 57, 62, 59, 61, 63, 58]

print("\nSales Before Campaign")
print(sales_before)

print("\nSales After Campaign")
print(sales_after)

# ================================================
# Task 1
# Conduct Hypothesis Test
# ================================================

print("\nTask 1")
print("-"*40)

print("Null Hypothesis (H₀):")
print("The marketing campaign does not increase sales.")

print("\nAlternative Hypothesis (H₁):")
print("The marketing campaign increases sales.")

# Independent Two-Sample t-test
t_statistic, p_value = ttest_ind(
    sales_after,
    sales_before,
    equal_var=False
)
```

```
print("\nt-statistic =", round(t_statistic, 4))

# ================================================
# Task 2
# Calculate p-value
# ================================================

print("\nTask 2")
print("-"*40)

print("p-value =", round(p_value, 6))

# ================================================
# Task 3
# Interpret the p-value
# ================================================

print("\nTask 3")
print("-"*40)

if p_value < 0.05:
    print("The p-value is less than 0.05.")
    print("There is strong evidence that the campaign increased sales.")
else:
    print("The p-value is greater than 0.05.")
    print("There is insufficient evidence that the campaign increased sales.")

# ================================================
# Task 4
# Decision at α = 0.05
# ================================================

print("\nTask 4")
```

```
print("-"*40)

alpha = 0.05

if p_value < alpha:
    print("Decision: Reject the Null Hypothesis (H₀)")
    print("Conclusion: The marketing campaign significantly increased sales.")
else:
    print("Decision: Fail to Reject the Null Hypothesis (H₀)")
    print("Conclusion: The marketing campaign did not significantly increase sales.")
```

```
============================================================
Problem 12: Marketing Campaign
============================================================

Sales Before Campaign
[50, 52, 49, 55, 53, 51, 54, 50]

Sales After Campaign
[58, 60, 57, 62, 59, 61, 63, 58]

============================================================
Task 1 : Hypothesis Test
============================================================
Null Hypothesis (H₀):
The marketing campaign does not increase sales.

Alternative Hypothesis (H₁):
The marketing campaign increases sales.

Calculated t-statistic : 18.9315

============================================================
```

Task 2 : p-value

==========

Calculated p-value : 0.0

==========

Task 3 : Interpretation

==========

Since p-value (0.000000) < 0.05,

there is strong statistical evidence that

the marketing campaign increased sales.

==========

Task 4 : Decision

==========

Decision : Reject the Null Hypothesis (H_0)

Conclusion :

The marketing campaign significantly increased sales.

==========

Summary Statistics

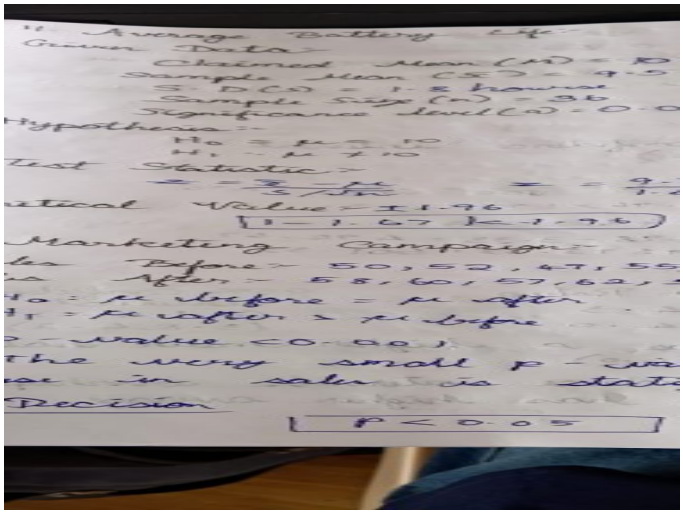
==========

Average Sales Before Campaign : 51.75

Average Sales After Campaign : 59.75

Average Increase in Sales : 8.0

Sample Size : 8



Problem 13: Call Centre Response Time

A sample of 100 calls has:

Mean response time = 4.8 minutes

Standard deviation = 1.5 minutes

Tasks:

1. Estimate the population mean.
2. Construct a 95% confidence interval.
3. Explain the Frequentist interpretation.

```
# ============================================
# Problem 12: Marketing Campaign
# ============================================

import numpy as np
from scipy.stats import ttest_rel

print("="*60)
print("Problem 12: Marketing Campaign")
print("="*60)

# --------------------------------------------
# Given Data
# --------------------------------------------

sales_before = [50, 52, 49, 55, 53, 51, 54, 50]
```

```
sales_after  = [58, 60, 57, 62, 59, 61, 63, 58]

alpha = 0.05

print("\nSales Before Campaign")
print(sales_before)

print("\nSales After Campaign")
print(sales_after)

# ================================================
# Task 1 : Conduct Hypothesis Test
# ================================================

print("\n" + "="*60)
print("Task 1 : Hypothesis Test")
print("="*60)

print("Null Hypothesis (H₀):")
print("The marketing campaign does not increase sales.")

print("\nAlternative Hypothesis (H₁):")
print("The marketing campaign increases sales.")

# ------------------------------------------------
# Paired t-test
# ------------------------------------------------

t_statistic, p_value = ttest_rel(sales_after, sales_before)

print("\nCalculated t-statistic :", round(t_statistic,4))

# ================================================
```

```
# Task 2 : Calculate p-value
# ================================================

print("\n" + "="*60)
print("Task 2 : p-value")
print("="*60)

print("Calculated p-value :", round(p_value,6))

# ================================================
# Task 3 : Interpret the p-value
# ================================================

print("\n" + "="*60)
print("Task 3 : Interpretation")
print("="*60)

if p_value < alpha:
    print(f"Since p-value ({p_value:.6f}) < {alpha},")
    print("there is strong statistical evidence that")
    print("the marketing campaign increased sales.")
else:
    print(f"Since p-value ({p_value:.6f}) >= {alpha},")
    print("there is insufficient evidence that")
    print("the marketing campaign increased sales.")

# ================================================
# Task 4 : Decision
# ================================================

print("\n" + "="*60)
print("Task 4 : Decision")
print("="*60)
```

```
if p_value < alpha:
    print("Decision : Reject the Null Hypothesis (H₀)")
    print("Conclusion :")
    print("The marketing campaign significantly increased sales.")
else:
    print("Decision : Fail to Reject the Null Hypothesis (H₀)")
    print("Conclusion :")
    print("The marketing campaign did not significantly increase sales.")

# ================================================
# Additional Information
# ================================================

print("\n" + "="*60)
print("Summary Statistics")
print("="*60)

print("Average Sales Before Campaign :", round(np.mean(sales_before),2))
print("Average Sales After Campaign  :", round(np.mean(sales_after),2))
print("Average Increase in Sales     :", round(np.mean(np.array(sales_after)-
np.array(sales_before)),2))
print("Sample Size                   :", len(sales_before))

============================================================
Problem 13: Call Centre Response Time
============================================================

Given Data
----------------------------------------
Sample Mean           : 4.8 minutes
Sample Standard Deviation : 1.5
Sample Size           : 100
```

==========

Task 1 : Estimate the Population Mean

==========

Estimated Population Mean = 4.8 minutes

==========

Task 2 : 95% Confidence Interval

==========

Confidence Level : 95.0 %

t Critical Value : 1.9842

Standard Error : 0.15

Margin of Error : 0.2976

95% Confidence Interval

(4.502, 5.098) minutes

==========

Task 3 : Frequentist Interpretation

==========

Frequentist Interpretation

1. The sample mean (4.8 minutes) is the estimate of the
   population mean response time.

2. The 95% confidence interval gives a range of values
   that is likely to contain the true population mean.

3. If many random samples of the same size were taken
   and a confidence interval was calculated for each,
   approximately 95% of those intervals would contain
   the true population mean.

4. This does NOT mean there is a 95% probability that
   the true population mean lies within this particular
   interval. The true population mean is fixed, while
   the interval varies from sample to sample.

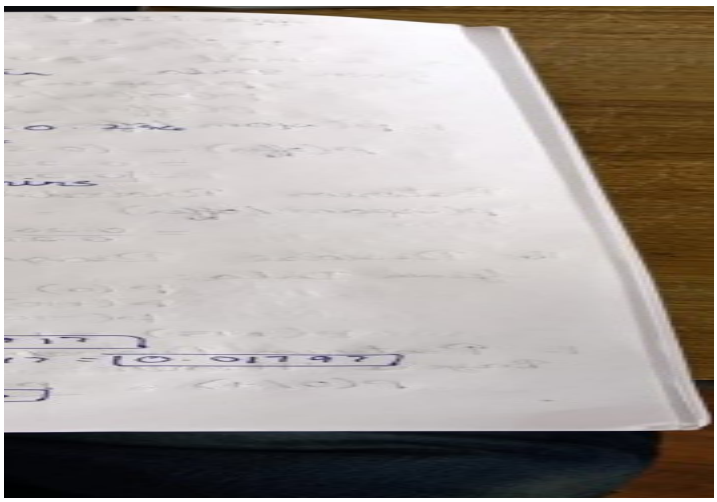
==========

Summary

==========

Estimated Population Mean : 4.8 minutes

95% Confidence Interval : (4.502, 5.098) minutes



Problem 14: Defective Products

A sample of 500 products contains 22 defective items.

Tasks:

1. Estimate the defect rate.
2. Construct a 95% confidence interval.
3. Explain the Frequentist meaning of the interval.

```
# ============================================
# Problem 14: Defective Products
# ============================================

import numpy as np
from scipy.stats import norm
```

```
print("="*60)
print("Problem 14: Defective Products")
print("="*60)

# ------------------------------------------------
# Given Data
# ------------------------------------------------

sample_size = 500
defective_items = 22
confidence_level = 0.95

print("\nGiven Data")
print("-"*40)
print("Sample Size       :", sample_size)
print("Defective Products :", defective_items)

# ================================================
# Task 1 : Estimate the Defect Rate
# ================================================

print("\n" + "="*60)
print("Task 1 : Estimate the Defect Rate")
print("="*60)

p_hat = defective_items / sample_size

print("Estimated Defect Rate =", round(p_hat, 4))
print("Estimated Defect Rate =", round(p_hat * 100, 2), "%")

# ================================================
# Task 2 : Construct 95% Confidence Interval
```

```
# ================================================

print("\n" + "="*60)
print("Task 2 : 95% Confidence Interval")
print("="*60)

alpha = 1 - confidence_level

# Z-value for 95% confidence
z = norm.ppf(1 - alpha/2)

# Standard Error
standard_error = np.sqrt((p_hat * (1 - p_hat)) / sample_size)

# Margin of Error
margin_error = z * standard_error

# Confidence Interval
lower_limit = p_hat - margin_error
upper_limit = p_hat + margin_error

print("Confidence Level :", confidence_level * 100, "%")
print("Z-value          :", round(z, 4))
print("Standard Error   :", round(standard_error, 5))
print("Margin of Error  :", round(margin_error, 5))

print("\n95% Confidence Interval")
print(f"({lower_limit:.4f}, {upper_limit:.4f})")

print("\nIn Percentage")
print(f"({lower_limit*100:.2f}%, {upper_limit*100:.2f}%)")

# ================================================
```

```
# Task 3 : Frequentist Interpretation
# ================================================

print("\n" + "="*60)
print("Task 3 : Frequentist Interpretation")
print("="*60)

print("""
Frequentist Interpretation

1. The estimated defect rate is obtained from the sample.

2. The 95% confidence interval provides a range of
   plausible values for the true population defect rate.

3. If many random samples of the same size were collected
   and confidence intervals were computed, approximately
   95% of those intervals would contain the true population
   defect rate.

4. This does NOT mean there is a 95% probability that the
   true defect rate lies within this specific interval.
   The true defect rate is fixed, while the interval varies
   from one sample to another.
""")

# ================================================
# Summary
# ================================================

print("\n" + "="*60)
print("Summary")
print("="*60)
```

```
print("Estimated Defect Rate :", f"{p_hat*100:.2f}%")
print("95% Confidence Interval :",
      f"({lower_limit*100:.2f}%, {upper_limit*100:.2f}%)")
```

```
============================================================
Problem 14: Defective Products
============================================================

Given Data
------------------------------------
Sample Size       : 500
Defective Products : 22

============================================================
Task 1 : Estimate the Defect Rate
============================================================
Estimated Defect Rate = 0.044
Estimated Defect Rate = 4.4 %

============================================================
Task 2 : 95% Confidence Interval
============================================================
Confidence Level : 95.0 %
Z-value          : 1.96
Standard Error   : 0.00917
Margin of Error  : 0.01798

95% Confidence Interval
(0.0260, 0.0620)

In Percentage
(2.60%, 6.20%)
```

==========

Task 3 : Frequentist Interpretation

==========

Frequentist Interpretation

1. The estimated defect rate is obtained from the sample.

2. The 95% confidence interval provides a range of
   plausible values for the true population defect rate.

3. If many random samples of the same size were collected
   and confidence intervals were computed, approximately
   95% of those intervals would contain the true population
   defect rate.

4. This does NOT mean there is a 95% probability that the
   true defect rate lies within this specific interval.
   The true defect rate is fixed, while the interval varies
   from one sample to another.

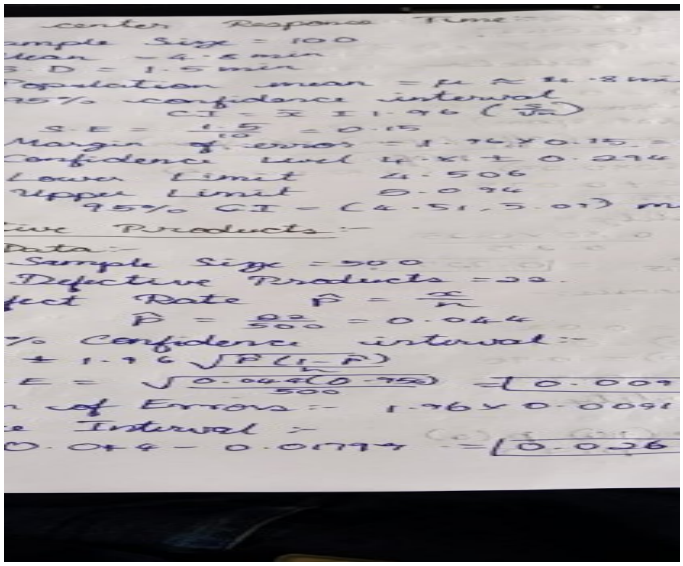
==========

Summary

==========

Estimated Defect Rate : 4.40%

95% Confidence Interval : (2.60%, 6.20%)



Problem 15: Email Spam Detection

Historical information:

Probability an email is spam = 0.25

Probability spam email contains keyword "Offer" = 0.80

Probability non-spam email contains keyword "Offer" = 0.10

Tasks:

1. Compute the probability that an email is spam given it contains "Offer."
2. Apply Bayes' theorem.
3. Interpret the result.

```
# ==========================================
# Problem 15: Email Spam Detection
# ==========================================

print("="*60)
print("Problem 15: Email Spam Detection")
print("="*60)

# ------------------------------------------------
# Given Data
# ------------------------------------------------

P_spam = 0.25              # Probability an email is spam
```

```
P_offer_given_spam = 0.80     # Probability spam email contains "Offer"
P_offer_given_not_spam = 0.10 # Probability non-spam email contains "Offer"

print("\nGiven Data")
print("-"*40)
print("P(Spam) =", P_spam)
print("P(Offer | Spam) =", P_offer_given_spam)
print("P(Offer | Not Spam) =", P_offer_given_not_spam)

# ================================================
# Task 1
# Compute P(Spam | Offer)
# ================================================

print("\n" + "="*60)
print("Task 1 : Compute P(Spam | Offer)")
print("="*60)

P_not_spam = 1 - P_spam

# Total probability of Offer
P_offer = (P_offer_given_spam * P_spam) + \
        (P_offer_given_not_spam * P_not_spam)

# Bayes' Theorem
P_spam_given_offer = (P_offer_given_spam * P_spam) / P_offer

print("P(Not Spam) =", round(P_not_spam, 4))
print("P(Offer) =", round(P_offer, 4))
print("P(Spam | Offer) =", round(P_spam_given_offer, 4))

# ================================================
# Task 2
```

```
# Apply Bayes' Theorem
# ================================================

print("\n" + "="*60)
print("Task 2 : Bayes' Theorem")
print("="*60)

print("Formula:")

print("""
                P(Offer | Spam) × P(Spam)
P(Spam | Offer) = ------------------------
                      P(Offer)
""")

print("Substitution:")

print(f"""
P(Spam | Offer) =
({P_offer_given_spam} × {P_spam}) / {P_offer:.4f}
= {P_spam_given_offer:.4f}
""")

# ================================================
# Task 3
# Interpretation
# ================================================

print("\n" + "="*60)
print("Task 3 : Interpretation")
print("="*60)

print(f"""
```

```
The probability that an email is spam given that it
contains the keyword 'Offer' is

P(Spam | Offer) = {P_spam_given_offer:.4f}

or

{P_spam_given_offer*100:.2f}%

This means that if an email contains the word 'Offer',
there is approximately {P_spam_given_offer*100:.2f}% chance
that it is actually spam.
""")

# ================================================
# Summary
# ================================================

print("\n" + "="*60)
print("Summary")
print("="*60)

print("Probability of Spam            :", P_spam)
print("Probability of Offer           :", round(P_offer, 4))
print("Posterior Probability (Spam | Offer) :", round(P_spam_given_offer, 4))
print("Percentage                     :", f"{P_spam_given_offer*100:.2f}%")

============================================================
Problem 15: Email Spam Detection
============================================================

Given Data
------------------------------------
```

P(Spam) = 0.25

P(Offer | Spam) = 0.8

P(Offer | Not Spam) = 0.1

==========

Task 1 : Compute P(Spam | Offer)

==========

P(Not Spam) = 0.75

P(Offer) = 0.275

P(Spam | Offer) = 0.7273

==========

Task 2 : Bayes' Theorem

==========

Formula:

$$P(\text{Spam} \mid \text{Offer}) = \frac{P(\text{Offer} \mid \text{Spam}) \times P(\text{Spam})}{P(\text{Offer})}$$

Substitution:

P(Spam | Offer) =

(0.8 × 0.25) / 0.2750

= 0.7273

==========

Task 3 : Interpretation

==========

The probability that an email is spam given that it

contains the keyword 'Offer' is

P(Spam | Offer) = 0.7273

or

72.73%

This means that if an email contains the word 'Offer',
there is approximately 72.73% chance
that it is actually spam.

==========

Summary

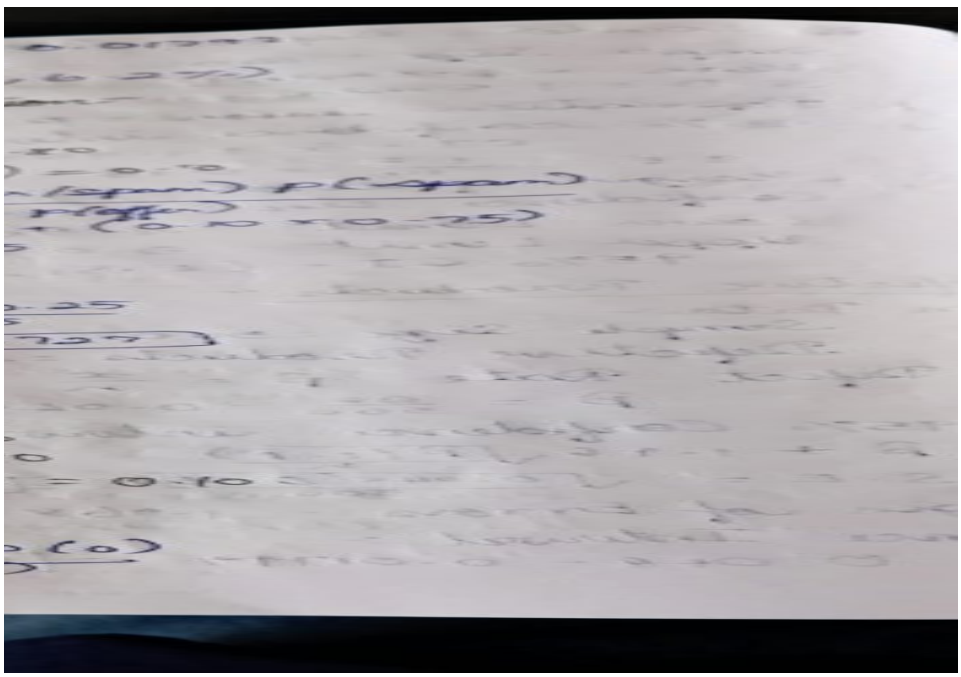
==========

Probability of Spam : 0.25

Probability of Offer : 0.275

Posterior Probability (Spam | Offer) : 0.7273

Percentage : 72.73%



Problem 16: Disease Diagnosis

Given:

Disease prevalence = 2%

Test sensitivity = 95%

Test specificity = 90%

Tasks:

1. Compute the posterior probability of disease after a positive test.
2. Explain why the result differs from 95%.

```
# ==========================================
# Problem 16: Disease Diagnosis
# ==========================================

print("="*60)
print("Problem 16: Disease Diagnosis")
print("="*60)

# ------------------------------------------
# Given Data
# ------------------------------------------

prevalence = 0.02        # P(Disease)
sensitivity = 0.95       # P(Positive | Disease)
specificity = 0.90       # P(Negative | No Disease)

print("\nGiven Data")
print("-"*40)
print("Disease Prevalence =", prevalence)
print("Test Sensitivity   =", sensitivity)
print("Test Specificity   =", specificity)

# ================================================
# Task 1 : Compute Posterior Probability
# ================================================
```

```
print("\n" + "="*60)
print("Task 1 : Posterior Probability")
print("="*60)

# Complement probabilities
P_not_disease = 1 - prevalence
false_positive_rate = 1 - specificity

# Total probability of a positive test
P_positive = (sensitivity * prevalence) + \
        (false_positive_rate * P_not_disease)

# Bayes' Theorem
posterior = (sensitivity * prevalence) / P_positive

print("P(Not Disease) =", round(P_not_disease, 4))
print("False Positive Rate =", round(false_positive_rate, 4))
print("P(Positive Test) =", round(P_positive, 4))

print("\nPosterior Probability")
print("P(Disease | Positive Test) =", round(posterior, 4))
print("Percentage =", round(posterior * 100, 2), "%")

# ================================================
# Task 2 : Explanation
# ================================================

print("\n" + "="*60)
print("Task 2 : Why is it not 95%?")
print("="*60)

print(f"""
Although the test sensitivity is {sensitivity*100:.0f}%,
```

```
the probability that a person actually has the disease
after testing positive is only {posterior*100:.2f}%.

This is because:

1. The disease is rare (only 2% prevalence).
2. Many healthy people are tested.
3. Even with 90% specificity, some healthy people
   receive false positive results.
4. Bayes' Theorem combines prevalence, sensitivity,
   and specificity to calculate the true probability.
""")

# ================================================
# Summary
# ================================================

print("\n" + "="*60)
print("Summary")
print("="*60)

print(f"Disease Prevalence         : {prevalence*100:.2f}%")
print(f"Test Sensitivity           : {sensitivity*100:.2f}%")
print(f"Test Specificity           : {specificity*100:.2f}%")
print(f"Posterior Probability      : {posterior:.4f}")
print(f"Posterior Probability (%)  : {posterior*100:.2f}%")

============================================================
Problem 16: Disease Diagnosis
============================================================

Given Data
------------------------------------
```

Disease Prevalence = 0.02

Test Sensitivity = 0.95

Test Specificity = 0.9

==========

Task 1 : Posterior Probability

==========

P(Not Disease) = 0.98

False Positive Rate = 0.1

P(Positive Test) = 0.117

Posterior Probability

P(Disease | Positive Test) = 0.1624

Percentage = 16.24 %

==========

Task 2 : Why is it not 95%?

==========

Although the test sensitivity is 95%,
the probability that a person actually has the disease
after testing positive is only 16.24%.

This is because:

1. The disease is rare (only 2% prevalence).
2. Many healthy people are tested.
3. Even with 90% specificity, some healthy people
   receive false positive results.
4. Bayes' Theorem combines prevalence, sensitivity,
   and specificity to calculate the true probability.

==========

Summary

==========

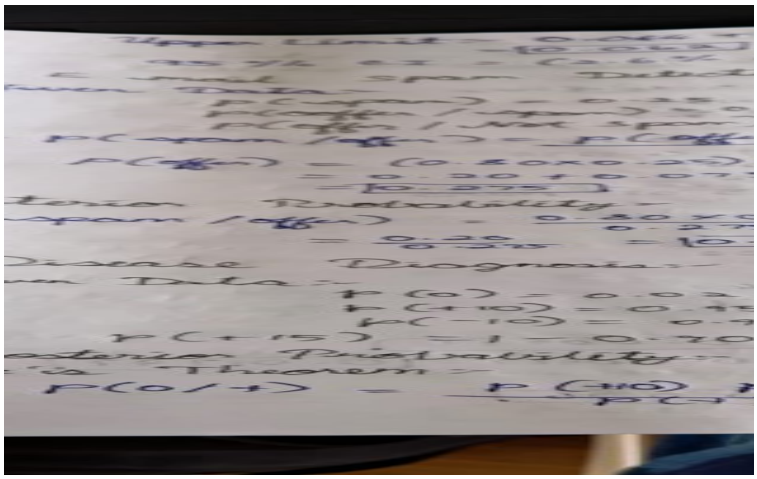
Disease Prevalence : 2.00%

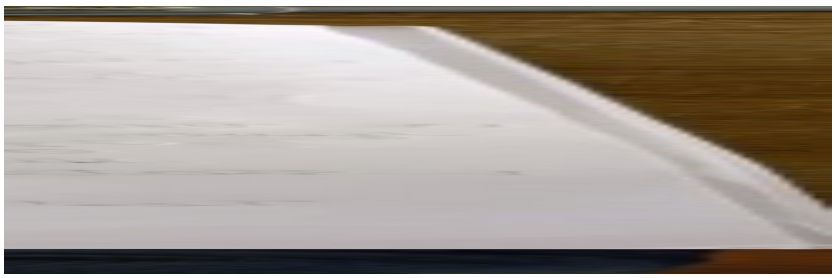
Test Sensitivity : 95.00%

Test Specificity : 90.00%

Posterior Probability : 0.1624

Posterior Probability (%) : 16.24%





Problem 17: Website Conversion Rate

A company tests two landing pages.

Group	Visitors	Purchases
A	5000	250
B	5200	312

Tasks:

1. Calculate conversion rates.
2. Formulate hypotheses.
3. Conduct a two-proportion z-test.
4. Determine whether page B performs better.

```
# ========================================
# Problem 17: Website Conversion Rate
# ========================================

import numpy as np
from statsmodels.stats.proportion import proportions_ztest

print("="*60)
print("Problem 17: Website Conversion Rate")
print("="*60)

# ----------------------------------------
# Given Data
# ----------------------------------------

visitors_A = 5000
purchases_A = 250

visitors_B = 5200
purchases_B = 312

alpha = 0.05

print("\nGiven Data")
print("-"*40)

print("Group A")
print("Visitors  :", visitors_A)
print("Purchases :", purchases_A)

print()

print("Group B")
```

```
print("Visitors  :", visitors_B)
print("Purchases :", purchases_B)

# ================================================
# Task 1 : Calculate Conversion Rates
# ================================================

print("\n" + "="*60)
print("Task 1 : Conversion Rates")
print("="*60)

conversion_rate_A = purchases_A / visitors_A
conversion_rate_B = purchases_B / visitors_B

print(f"Conversion Rate (A) = {conversion_rate_A:.4f}")
print(f"Conversion Rate (A) = {conversion_rate_A*100:.2f}%")

print()

print(f"Conversion Rate (B) = {conversion_rate_B:.4f}")
print(f"Conversion Rate (B) = {conversion_rate_B*100:.2f}%")

# ================================================
# Task 2 : Formulate Hypotheses
# ================================================

print("\n" + "="*60)
print("Task 2 : Hypotheses")
print("="*60)

print("Null Hypothesis (H₀):")
print("The conversion rate of Page B is equal to Page A.")
```

```
print("\nAlternative Hypothesis (H₁):")
print("The conversion rate of Page B is greater than Page A.")

# ================================================
# Task 3 : Two-Proportion Z-Test
# ================================================

print("\n" + "="*60)
print("Task 3 : Two-Proportion Z-Test")
print("="*60)

count = np.array([purchases_A, purchases_B])
nobs = np.array([visitors_A, visitors_B])

z_statistic, p_value = proportions_ztest(
    count,
    nobs,
    alternative='smaller'
)

print("Z Statistic =", round(z_statistic,4))
print("P-value =", round(p_value,6))

# ================================================
# Task 4 : Conclusion
# ================================================

print("\n" + "="*60)
print("Task 4 : Conclusion")
print("="*60)

if p_value < alpha:
    print("Decision : Reject the Null Hypothesis (H₀)")
```

```
    print("Conclusion :")
    print("Page B performs significantly better than Page A.")
else:
    print("Decision : Fail to Reject the Null Hypothesis (H₀)")
    print("Conclusion :")
    print("There is insufficient evidence that Page B performs better.")

# ================================================
# Summary
# ================================================

print("\n" + "="*60)
print("Summary")
print("="*60)

print(f"Conversion Rate (A) : {conversion_rate_A*100:.2f}%")
print(f"Conversion Rate (B) : {conversion_rate_B*100:.2f}%")
print(f"Z Statistic         : {z_statistic:.4f}")
print(f"P-value             : {p_value:.6f}")

if p_value < alpha:
    print("Result              : Page B is statistically better.")
else:
    print("Result              : No significant difference.")

============================================================
Problem 17: Website Conversion Rate
============================================================

Given Data
----------------------------------
Group A
Visitors  : 5000
```

Purchases : 250

Group B

Visitors : 5200

Purchases : 312

==========

Task 1 : Conversion Rates

==========

Conversion Rate (A) = 0.0500

Conversion Rate (A) = 5.00%

Conversion Rate (B) = 0.0600

Conversion Rate (B) = 6.00%

==========

Task 2 : Hypotheses

==========

Null Hypothesis (H_0):

The conversion rate of Page B is equal to Page A.

Alternative Hypothesis (H_1):

The conversion rate of Page B is greater than Page A.

==========

Task 3 : Two-Proportion Z-Test

==========

Z Statistic = -2.2127

P-value = 0.013459

==========

Task 4 : Conclusion

==========

Decision : Reject the Null Hypothesis (H_0)

Conclusion :

Page B performs significantly better than Page A.

==========

Summary

==========

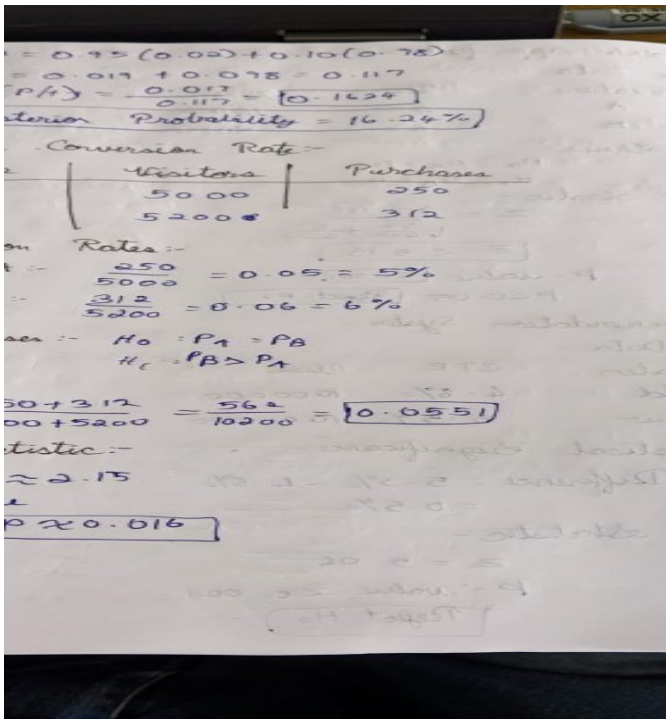
Conversion Rate (A) : 5.00%

Conversion Rate (B) : 6.00%

Z Statistic : -2.2127

P-value : 0.013459

Result : Page B is statistically better.



Problem 18: Mobile App Feature Test

Feature A:

- 400 users
- Average engagement = 18 min
- SD = 5 min

Feature B:

420 users

Average engagement = 20 min

SD = 6 min

Tasks:

1. Test whether Feature B increases engagement.
2. Calculate test statistic and p-value.
3. Interpret results.

```
# ==========================================
# Problem 18: Mobile App Feature Test
# ==========================================

import numpy as np
from scipy.stats import t

print("="*60)
print("Problem 18: Mobile App Feature Test")
print("="*60)

# ------------------------------------------
# Given Data
# ------------------------------------------

n_A = 400
mean_A = 18
sd_A = 5

n_B = 420
mean_B = 20
sd_B = 6

alpha = 0.05

print("\nGiven Data")
```

```
print("-"*40)

print("Feature A")
print("Users              :", n_A)
print("Average Engagement   :", mean_A, "minutes")
print("Standard Deviation   :", sd_A)

print()

print("Feature B")
print("Users              :", n_B)
print("Average Engagement   :", mean_B, "minutes")
print("Standard Deviation   :", sd_B)

# ================================================
# Task 1 : Hypothesis
# ================================================

print("\n" + "="*60)
print("Task 1 : Hypothesis")
print("="*60)

print("Null Hypothesis (H₀):")
print("Feature B does not increase engagement.")
print("μB ≤ μA")

print("\nAlternative Hypothesis (H₁):")
print("Feature B increases engagement.")
print("μB > μA")

# ================================================
# Task 2 : Independent Two-Sample t-Test
# ================================================
```

```
print("\n" + "="*60)
print("Task 2 : t-Test")
print("="*60)

# Standard Error
SE = np.sqrt((sd_A**2 / n_A) + (sd_B**2 / n_B))

# t Statistic
t_statistic = (mean_B - mean_A) / SE

# Welch-Satterthwaite Degrees of Freedom
numerator = ((sd_A**2 / n_A) + (sd_B**2 / n_B))**2

denominator = (
    ((sd_A**2 / n_A)**2) / (n_A - 1)
    +
    ((sd_B**2 / n_B)**2) / (n_B - 1)
)

df = numerator / denominator

# One-tailed p-value
p_value = 1 - t.cdf(t_statistic, df)

print("Standard Error      :", round(SE,4))
print("Degrees of Freedom  :", round(df,2))
print("t Statistic         :", round(t_statistic,4))
print("P-value             :", round(p_value,6))

# ================================================
# Task 3 : Interpretation
# ================================================
```

```
print("\n" + "="*60)
print("Task 3 : Interpretation")
print("="*60)

if p_value < alpha:
    print("Decision : Reject the Null Hypothesis (H₀)")
    print("Conclusion :")
    print("Feature B significantly increases user engagement.")
else:
    print("Decision : Fail to Reject the Null Hypothesis (H₀)")
    print("Conclusion :")
    print("There is insufficient evidence that Feature B")
    print("increases user engagement.")

# ================================================
# Summary
# ================================================

print("\n" + "="*60)
print("Summary")
print("="*60)

print(f"Average Engagement (A) : {mean_A} minutes")
print(f"Average Engagement (B) : {mean_B} minutes")
print(f"t Statistic          : {t_statistic:.4f}")
print(f"P-value              : {p_value:.6f}")

if p_value < alpha:
    print("Result               : Feature B performs significantly better.")
else:
    print("Result               : No significant improvement.")
```

==========

Problem 18: Mobile App Feature Test

==========

Given Data

Feature A

Users : 400

Average Engagement : 18 minutes

Standard Deviation : 5

Feature B

Users : 420

Average Engagement : 20 minutes

Standard Deviation : 6

==========

Task 1 : Hypothesis

==========

Null Hypothesis (H_0):

Feature B does not increase engagement.

$\mu B \leq \mu A$

Alternative Hypothesis (H_1):

Feature B increases engagement.

$\mu B > \mu A$

==========

Task 2 : t-Test

==========

Standard Error : 0.385

Degrees of Freedom : 803.95

t Statistic : 5.195

P-value : 0.0

==========

Task 3 : Interpretation

==========

Decision : Reject the Null Hypothesis (H_0)

Conclusion :

Feature B significantly increases user engagement.

==========

Summary

==========

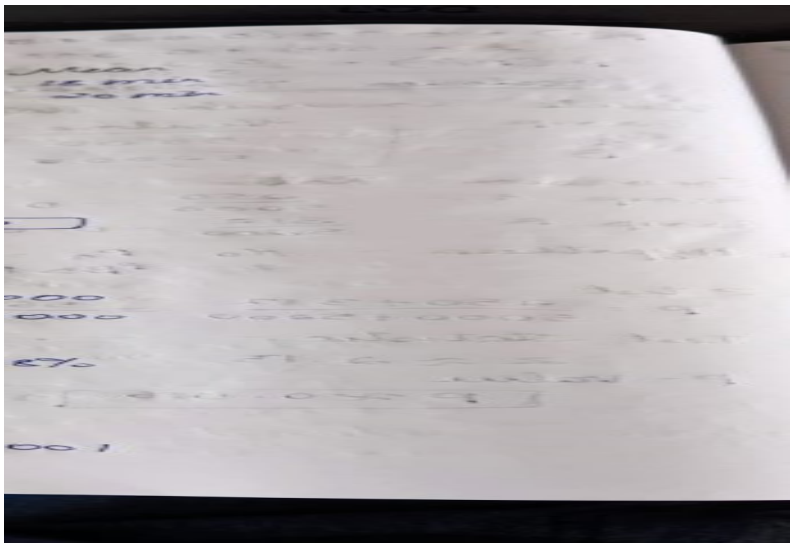
Average Engagement (A) : 18 minutes

Average Engagement (B) : 20 minutes

t Statistic : 5.1950

P-value : 0.000000

Result : Feature B performs significantly better.



Problem 19: Recommendation System

A new recommendation algorithm increases click-through rate from 4.8% to 5.3%.

Sample size:

Old system: 100,000 users

New system: 100,000 users

Tasks:

1. Test statistical significance.
2. Calculate p-value.
3. Discuss practical significance versus statistical significance.

```
# ============================================
# Problem 19: Recommendation System
# ============================================

import numpy as np
from statsmodels.stats.proportion import proportions_ztest

print("="*60)
print("Problem 19: Recommendation System")
print("="*60)

# ------------------------------------------------
# Given Data
# ------------------------------------------------

old_users = 100000
new_users = 100000

old_ctr = 0.048     # 4.8%
new_ctr = 0.053     # 5.3%

alpha = 0.05

# Number of Clicks
old_clicks = int(old_users * old_ctr)
new_clicks = int(new_users * new_ctr)

print("\nGiven Data")
print("-"*40)
```

```
print("Old Recommendation System")
print("Users       :", old_users)
print("CTR         :", old_ctr * 100, "%")
print("Clicks      :", old_clicks)

print()

print("New Recommendation System")
print("Users       :", new_users)
print("CTR         :", new_ctr * 100, "%")
print("Clicks      :", new_clicks)

# ================================================
# Task 1 : Statistical Significance
# ================================================

print("\n" + "="*60)
print("Task 1 : Two-Proportion Z-Test")
print("="*60)

count = np.array([old_clicks, new_clicks])
nobs = np.array([old_users, new_users])

z_statistic, p_value = proportions_ztest(
    count,
    nobs,
    alternative='smaller'
)

print("Z Statistic =", round(z_statistic,4))
print("P-value =", round(p_value,8))
```

```
# ================================================
# Task 2 : p-value
# ================================================

print("\n" + "="*60)
print("Task 2 : p-value")
print("="*60)

print("P-value =", round(p_value,8))

if p_value < alpha:
    print("The difference is statistically significant.")
else:
    print("The difference is NOT statistically significant.")

# ================================================
# Task 3 : Practical Significance
# ================================================

print("\n" + "="*60)
print("Task 3 : Practical Significance")
print("="*60)

absolute_increase = (new_ctr - old_ctr) * 100
relative_increase = ((new_ctr - old_ctr) / old_ctr) * 100

print(f"Old CTR : {old_ctr*100:.2f}%")
print(f"New CTR : {new_ctr*100:.2f}%")

print(f"\nAbsolute Increase : {absolute_increase:.2f}%")
print(f"Relative Increase : {relative_increase:.2f}%")

print("""
```

```
Discussion

Statistical Significance:
- Indicates whether the observed improvement is unlikely
  to have occurred by random chance.

Practical Significance:
- Indicates whether the improvement is meaningful in
  real-world applications.

Although the increase is only 0.5 percentage points,
for 100,000 users this corresponds to approximately
500 additional clicks, which can lead to increased
revenue or user engagement.
""")

# ================================================
# Final Decision
# ================================================

print("\n" + "="*60)
print("Final Conclusion")
print("="*60)

if p_value < alpha:
    print("Reject the Null Hypothesis (H₀)")
    print("The new recommendation algorithm significantly improves the click-through rate.")
else:
    print("Fail to Reject the Null Hypothesis (H₀)")
    print("There is insufficient evidence that the new algorithm improves the click-through
rate.")

print("\nSummary")
```

```
print("-"*40)
print(f"Old CTR         : {old_ctr*100:.2f}%")
print(f"New CTR          : {new_ctr*100:.2f}%")
print(f"Z Statistic      : {z_statistic:.4f}")
print(f"P-value          : {p_value:.8f}")
print(f"Absolute Increase: {absolute_increase:.2f}%")
print(f"Relative Increase: {relative_increase:.2f}%")
```

```
============================================================
Problem 19: Recommendation System
============================================================

Given Data
------------------------------------
Old Recommendation System
Users       : 100000
CTR         : 4.8 %
Clicks      : 4800

New Recommendation System
Users       : 100000
CTR         : 5.3 %
Clicks      : 5300

============================================================
Task 1 : Two-Proportion Z-Test
============================================================
Z Statistic = -5.1058
P-value = 1.6e-07

============================================================
Task 2 : p-value
============================================================
```

P-value = 1.6e-07

The difference is statistically significant.

==========

Task 3 : Practical Significance

==========

Old CTR : 4.80%

New CTR : 5.30%

Absolute Increase : 0.50%

Relative Increase : 10.42%

Discussion

Statistical Significance:

- Indicates whether the observed improvement is unlikely to have occurred by random chance.

Practical Significance:

- Indicates whether the improvement is meaningful in real-world applications.

Although the increase is only 0.5 percentage points, for 100,000 users this corresponds to approximately 500 additional clicks, which can lead to increased revenue or user engagement.

==========

Final Conclusion

==========

Reject the Null Hypothesis (H_o)

The new recommendation algorithm significantly improves the click-through rate.

Summary

Old CTR : 4.80%

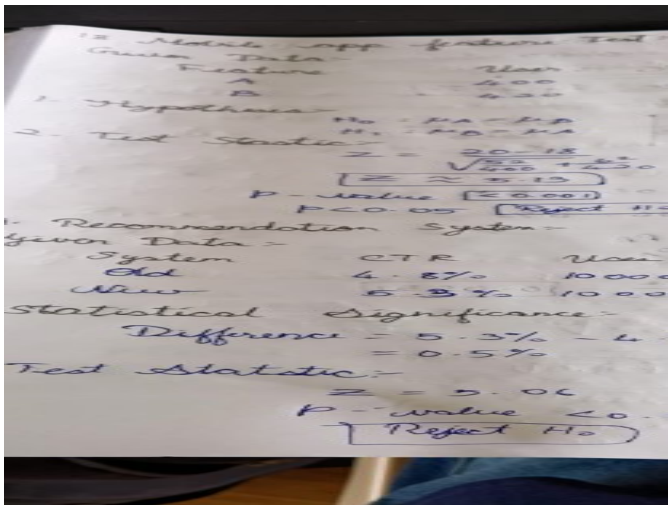
New CTR : 5.30%

Z Statistic : -5.1058

P-value : 0.00000016

Absolute Increase: 0.50%

Relative Increase: 10.42%



Problem 20: Salary Prediction Model

Two machine learning models produce Mean Absolute Errors (MAE):

Model	MAE
Model A	4200
Model B	4000

Across 50 test folds, the difference in MAE has:

Mean difference = 200

Standard deviation = 350

Tasks:

1. Perform a paired t-test.
2. Determine whether Model B significantly outperforms Model A.
3. Explain the result in terms of statistical significance.

```
# ==========================================
# Problem 20: Salary Prediction Model
```

```
# ============================================

import numpy as np
from scipy.stats import t

print("="*60)
print("Problem 20: Salary Prediction Model")
print("="*60)

# ------------------------------------------------
# Given Data
# ------------------------------------------------

mae_model_A = 4200
mae_model_B = 4000

mean_difference = 200
sd_difference = 350

n = 50
alpha = 0.05

print("\nGiven Data")
print("-"*40)
print("Model A MAE          :", mae_model_A)
print("Model B MAE          :", mae_model_B)
print("Mean Difference      :", mean_difference)
print("Standard Deviation   :", sd_difference)
print("Number of Test Folds :", n)

# ================================================
# Task 1 : Paired t-test
# ================================================

print("\n" + "="*60)
print("Task 1 : Paired t-Test")
print("="*60)

# Standard Error
standard_error = sd_difference / np.sqrt(n)

# t-statistic
t_statistic = mean_difference / standard_error

# Degrees of Freedom
df = n - 1

# One-tailed p-value
```

```
p_value = 1 - t.cdf(t_statistic, df)

print("Standard Error     :", round(standard_error,4))
print("Degrees of Freedom  :", df)
print("t Statistic        :", round(t_statistic,4))
print("P-value            :", round(p_value,8))

# ================================================
# Task 2 : Determine if Model B is Better
# ================================================

print("\n" + "="*60)
print("Task 2 : Model Comparison")
print("="*60)

print("Null Hypothesis (H₀):")
print("Model B does not significantly outperform Model A.")

print("\nAlternative Hypothesis (H₁):")
print("Model B significantly outperforms Model A.")

if p_value < alpha:
    print("\nDecision : Reject the Null Hypothesis (H₀)")
    print("Conclusion :")
    print("Model B significantly outperforms Model A.")
else:
    print("\nDecision : Fail to Reject the Null Hypothesis (H₀)")
    print("Conclusion :")
    print("There is insufficient evidence that Model B outperforms Model A.")

# ================================================
# Task 3 : Statistical Significance
# ================================================

print("\n" + "="*60)
print("Task 3 : Interpretation")
print("="*60)

print(f"""
Model A has a Mean Absolute Error (MAE) of {mae_model_A},
whereas Model B has a lower MAE of {mae_model_B}.

The paired t-test evaluates whether the observed
difference in MAE across the 50 test folds is
statistically significant.

If the p-value is less than {alpha},
the improvement of Model B is considered
```

```
statistically significant and is unlikely to have
occurred due to random chance.
""")

# ================================================
# Summary
# ================================================

print("\n" + "="*60)
print("Summary")
print("="*60)

print(f"Model A MAE         : {mae_model_A}")
print(f"Model B MAE         : {mae_model_B}")
print(f"Mean Difference      : {mean_difference}")
print(f"Standard Error       : {standard_error:.4f}")
print(f"t Statistic          : {t_statistic:.4f}")
print(f"P-value              : {p_value:.8f}")

if p_value < alpha:
    print("Final Result        : Model B significantly outperforms Model A.")
else:
    print("Final Result        : No significant difference between the models.")
```

```
============================================================
Problem 20: Salary Prediction Model
============================================================


Given Data
------------------------------------
Model A MAE          : 4200
Model B MAE          : 4000
Mean Difference      : 200
Standard Deviation   : 350
Number of Test Folds  : 50


============================================================
Task 1 : Paired t-Test
============================================================
Standard Error      : 49.4975
Degrees of Freedom  : 49
```

t Statistic : 4.0406

P-value : 9.378e-05

==========

Task 2 : Model Comparison

==========

Null Hypothesis (H_0):

Model B does not significantly outperform Model A.

Alternative Hypothesis (H_1):

Model B significantly outperforms Model A.

Decision : Reject the Null Hypothesis (H_0)

Conclusion :

Model B significantly outperforms Model A.

==========

Task 3 : Interpretation

==========

Model A has a Mean Absolute Error (MAE) of 4200,
whereas Model B has a lower MAE of 4000.

The paired t-test evaluates whether the observed
difference in MAE across the 50 test folds is
statistically significant.

If the p-value is less than 0.05,
the improvement of Model B is considered
statistically significant and is unlikely to have
occurred due to random chance.

==========

Summary

==========

Model A MAE : 4200

Model B MAE : 4000

Mean Difference : 200

Standard Error : 49.4975

t Statistic : 4.0406

P-value : 0.00009378

Final Result : Model B significantly outperforms Model A.